



Specifica Tecnica

The Bitless (Gruppo 18):

Lorenzo Battistella (2103116), Edoardo De Piccoli (2101055), Davide Facco (2087852),
Anna Marini (2110986), Andrea Menegaldo (2116426), Samuele Vendramin (2111934),
Simone Zecchinato (2113189)

Informazioni documento			
Versione	Data	Stato	Destinatari
0.5.0	2026-08-11	Stesura	The Bitless, prof. T. Vardanega, prof. R. Cardin, Zucchetti

Contatti: thebitless.swe@gmail.com

Registro delle modifiche					
Versione	Data	Autore	Verificatore	Approvatore	Descrizione
0.8.0	2026-08-11	A. Menegaldo		-	Inserimento diagrammi UML nella sezione Salvataggio e caricamento di una nota
0.7.0	2026-08-11	A. Menegaldo		-	Inserimento diagrammi UML nella sezione Architettura Frontend
0.6.0	2026-08-11	A. Menegaldo		-	Stesura sezioni Architettura Frontend
0.5.0	2026-08-11	A. Marini		-	Allineamento della sezione Tecnologie al repository di prodotto, stesura della Panoramica architetture e riorganizzazione dello scheletro delle sezioni di Architettura
0.4.0	2026-07-27	A. Marini	A. Menegaldo	-	Integrazione continua e analisi statica del backend, tracciamento dei requisiti nella sezione Tecnologie, riorganizzazione della struttura delle sezioni Architettura, Flusso dei dati e Tracciamento
0.3.0	2026-07-27	A. Marini	A. Menegaldo	-	Stesura della sezione Tecnologie
0.2.0	2026-07-27	A. Marini	A. Menegaldo	-	Riorganizzazione della struttura del documento e stesura della sezione Introduzione
0.1.0	2026-07-18	A. Menegaldo	A. Marini	-	Struttura del documento #1

Indice

1	Introduzione	5
1.1	Scopo del documento	5
1.2	Scopo del prodotto	5
1.3	Glossario	6
1.4	Riferimenti	6
1.4.1	Riferimenti normativi	6
1.4.2	Riferimenti informativi	6
2	Tecnologie	9
2.1	Criteri di scelta	9
2.2	Frontend	10
2.3	Backend	14
2.4	Persistenza delle note	17
2.5	Tecnologie di test e analisi	20
2.5.1	Analisi statica	20
2.5.2	Analisi dinamica	21
2.6	Infrastruttura	23
3	Architettura	27
3.1	Panoramica architetturale	27
3.1.1	Visione d'insieme	27
3.1.2	Stile architetturale del servizio applicativo	28
3.1.3	Stile architetturale dell'applicazione web	29
3.1.4	Vocabolario	29
3.2	Architettura di deployment	30
3.2.1	Unità di esecuzione e configurazione	30
3.2.2	Confronto con l'architettura a microservizi	30
3.2.3	Confronto con il monolite a strati	30
3.3	Architettura del frontend	30
3.3.1	Organizzazione dei sorgenti	31
3.3.2	Scomposizione in componenti	32
3.3.3	Gestione dello stato	38
3.3.4	Integrazione dell'editor	41
3.3.5	Accesso al servizio applicativo	44
3.3.6	Persistenza lato client	46
3.4	Architettura del backend	49
3.4.1	Struttura a esagono e contratti di dipendenza	49
3.4.2	Adattatori primari	49
3.4.3	Il nucleo	49
3.4.4	Adattatori secondari	49
3.4.5	Composition root e configurazione	49
3.5	Design pattern adottati	49
3.5.1	Object Adapter	49
3.5.2	Facade	49
3.5.3	Observer	49
3.5.4	Dependency Injection	49
3.5.5	Pattern valutati e non adottati	49

3.6	Idiomi e convenzioni di codifica	49
3.6.1	Generatori asincroni e propagazione dell'annullamento	49
3.6.2	Tipizzazione dei confini fra i livelli	49
3.6.3	Denominazione, struttura dei file e gestione degli errori	49
4	Flusso dei dati	49
4.1	Generazione assistita da LLM_G con risposta in streaming	49
4.2	Annullamento di un'operazione in corso	49
4.3	Generazione a partire da un collegamento	49
4.4	Salvataggio e caricamento di una nota	49
4.4.1	Persistenza su file system	49
4.4.2	Continuità della sessione	53
4.4.3	Recupero dei metadati dall'oggetto File	55
4.5	Propagazione e presentazione degli errori	56
5	Tracciamento dei requisiti$_G$	56
5.1	Criteri di tracciamento	56
5.2	Requisiti $_G$ funzionali	56
5.3	Requisiti $_G$ di qualità	56
5.4	Requisiti $_G$ di vincolo $_G$	56
5.5	Grafici riassuntivi	56

Elenco delle tabelle

1	Tecnologie adottate per il frontend	11
2	Tecnologie adottate per il backend	14
3	Strumenti di analisi statica	20
4	Strumenti di analisi dinamica	22
5	Tecnologie di infrastruttura	23
6	Vocabolario architetturale	29

1 Introduzione

1.1 Scopo del documento

Il presente documento descrive l'architettura del sistema_G **Second Brain**, prodotto realizzato dal gruppo **The Bitless** in risposta al capitolato_G **C6** proposto da **Zucchetti S.p.A.**. Mentre l'Analisi dei Requisiti_G stabilisce *che cosa* il sistema_G deve fare, la Specifica Tecnica stabilisce *come* il sistema_G è costruito per farlo.

Nello specifico, il documento persegue i seguenti obiettivi:

- **Motivazione delle scelte tecnologiche:** espone le tecnologie adottate per ciascun livello del prodotto, i bisogni tecnici che ne hanno determinato la selezione e le alternative valutate e scartate, con le relative motivazioni.
- **Descrizione dell'architettura:** illustra la scomposizione del sistema_G nelle sue componenti, le responsabilità attribuite a ciascuna di esse e le interazioni che ne regolano la collaborazione, sia in termini logici sia in termini di distribuzione delle parti in esecuzione.
- **Documentazione dei design pattern:** riporta i pattern architetturali e di progettazione adottati, motivandone l'impiego rispetto al problema specifico che risolvono all'interno del prodotto.
- **Tracciamento dei requisiti_G:** correla le componenti architetturali ai requisiti_G individuati nell'Analisi dei Requisiti_G, così da rendere verificabile la copertura del perimetro funzionale concordato con la proponente_G.

Il documento costituisce il riferimento progettuale per le successive fasi di realizzazione e di manutenzione del prodotto: ogni intervento sul codice deve risultare coerente con quanto qui descritto e, qualora introduca una variazione architetturale, deve essere preceduto dall'aggiornamento della presente specifica.

La Specifica Tecnica dà inoltre continuità al **Proof of Concept_G** sviluppato per la **Requirements and Technology Baseline_G**: le tecnologie e le soluzioni architetturali qui documentate sono quelle sperimentate e validate in quella sede, ora consolidate e portate al livello di dettaglio necessario allo sviluppo_G del prodotto completo.

Come gli altri documenti del gruppo, la stesura segue un approccio **incrementale_G**: il documento viene raffinato a ogni iterazione, in modo da riflettere costantemente lo stato effettivo del prodotto.

1.2 Scopo del prodotto

Il capitolato_G **C6** di **Zucchetti S.p.A.** richiede la realizzazione di **Second Brain**, un'applicazione web per la scrittura e la gestione di note in formato **Markdown_G**, assistita da modelli linguistici di grandi dimensioni (**LLM_G**).

L'interfaccia si articola in due pannelli affiancati: a sinistra un editor in cui l'utente compone il testo nella sintassi **Markdown_G**, a destra un'anteprima che ne mostra il rendering aggiornato in tempo reale. Su questa base si innestano le funzioni assistite dall'**LLM_G**, che operano sul testo selezionato o sull'intera nota: riassunto, riscrittura e adattamento del tono, correzione, traduzione multilingue, analisi critica secondo la tecnica dei **Sei Cappelli per Pensare_G** di Edward de Bono e generazione di testo secondo il paradigma del **Distant Writing_G**, in cui l'utente descrive sinteticamente il contenuto desiderato e delega al modello la stesura. Le note prodotte sono salvate e ricaricate sul file system locale dell'utente in formato **Markdown_G**.

Il valore del prodotto non risiede quindi nella sola redazione del testo, ma nel ruolo attivo assunto dal sistema_G, che affianca l'utente nelle fasi di ideazione, revisione e riorganizzazione dei contenuti.

Per la trattazione completa del perimetro funzionale, dei casi d'uso_G e della classificazione dei requisiti_G in obbligatori, desiderabili e opzionali si rimanda all'*Analisi dei Requisiti*, di cui il presente documento non intende costituire una duplicazione.

1.3 Glossario

La creazione e lo sviluppo_G di un sistema_G software richiedono una complessa operazione di progettazione e analisi del dominio_G, attività che deve necessariamente precedere la stesura del codice. Il gruppo si impegna a raccogliere e organizzare tali informazioni in modo che siano facilmente accessibili, al fine di favorire la massima asincronia ed efficienza nelle attività di progetto.

La principale fonte di ambiguità nello svolgimento delle attività è rappresentata dall'incomprensione dei termini tecnici o specialistici adottati. A tale scopo, la nomenclatura di riferimento viene raccolta nel *glossario*, un documento incrementale_G volto a definire ogni termine rilevante per il dominio_G del progetto.

Il gruppo si impegna a contrassegnare ogni occorrenza di un termine presente nel glossario mediante l'apposizione di una lettera **G** a pedice, come esemplificato di seguito:
parola_G

Al fine di garantire una corretta comprensione del dominio_G, è fondamentale che ogni membro del gruppo consulti periodicamente il glossario, assicurando così il costante allineamento sulle definizioni e sui concetti chiave.

1.4 Riferimenti

1.4.1 Riferimenti normativi

- **Capitolato C6 - Second Brain:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
Ultimo accesso: 20 maggio 2026.
- **Regolamento del Progetto Didattico:**
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
Ultimo accesso: 23 maggio 2026.
- **Norme di Progetto v1.0.0:**
https://thebitless.live/RTB/doc_interna/norme-di-progetto/norme-di-progetto.pdf
Ultimo accesso: 8 giugno 2026.

1.4.2 Riferimenti informativi

Documentazione di progetto

- **Glossario v1.0.0:**
https://thebitless.live/RTB/doc_interna/glossario/glossario.pdf
Ultimo accesso: 12 giugno 2026.

- **Analisi dei Requisiti v2.0.0:**
https://thebitless.live/RTB/doc_esterna/analisi-dei-requisiti/analisi-dei-requisiti.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-12 – Scelta delle tecnologie per il PoC_G:**
https://thebitless.live/RTB/doc_interna/verbali/VI_2026-05-12_quarto-sprint/VI_2026-05-12_quarto-sprint.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale interno VI_2026-05-20 – Progettazione del PoC_G:**
https://thebitless.live/RTB/doc_interna/verbali/VI_2026-05-20_progettazione-PoC/VI_2026-05-20_progettazione-PoC.pdf
Ultimo accesso: 27 luglio 2026.
- **Verbale esterno VE_2026-05-14 – Approvazione delle tecnologie da parte della proponente_G:**
https://thebitless.live/RTB/doc_esterna/verbali/VE_2026-05-14_AdR-scelta-tecnologie/VE_2026-05-14_AdR-scelta-tecnologie.pdf
Ultimo accesso: 27 luglio 2026.

Documentazione delle tecnologie adottate Documentazione ufficiale delle tecnologie descritte nella sezione 2. *Ultimo accesso* a tutte le risorse elencate di seguito: 27 luglio 2026.

- **TypeScript:** <https://www.typescriptlang.org/docs/>
- **React:** <https://react.dev/>
- **Zustand:** <https://zustand.docs.pmnd.rs/>
- **react-markdown:** <https://github.com/remarkjs/react-markdown>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **shadcn/ui:** <https://ui.shadcn.com/>
- **Radix UI Primitives:** <https://www.radix-ui.com/primitives>
- **CodeMirror 6:** <https://codemirror.net/docs/>
- **Vite:** <https://vite.dev/>
- **Python 3.12:** <https://docs.python.org/3.12/>
- **FastAPI:** <https://fastapi.tiangolo.com/>
- **Pydantic:** <https://docs.pydantic.dev/>
- **httpx:** <https://www.python-httpx.org/>
- **Uvicorn:** <https://www.uvicorn.org/>
- **Tavily:** <https://docs.tavily.com/>
- **LiteLLM:** <https://docs.litellm.ai/>
- **Server-Sent Events (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
- **File System Access API (MDN):** https://developer.mozilla.org/en-US/docs/Web/API/File_System_API
- **Specifica OpenAPI:** <https://spec.openapis.org/oas/latest.html>

- **openapi-typescript**: <https://openapi-ts.dev/>
- **ESLint**: <https://eslint.org/docs/latest/>
- **Ruff**: <https://docs.astral.sh/ruff/>
- **mypy**: <https://mypy.readthedocs.io/>
- **Vitest**: <https://vitest.dev/>
- **Copertura del codice in Vitest**: <https://vitest.dev/guide/coverage>
- **Testing Library**: <https://testing-library.com/docs/react-testing-library/intro/>
- **pytest**: <https://docs.pytest.org/>
- **Docker**: <https://docs.docker.com/>
- **Docker Compose**: <https://docs.docker.com/compose/>
- **GitHub Actions**: <https://docs.github.com/actions>
- **pnpm**: <https://pnpm.io/>
- **uv**: <https://docs.astral.sh/uv/>
- **Node.js – calendario di supporto delle versioni**: <https://github.com/nodejs/release>
- **File API – interfaccia File (MDN)**: <https://developer.mozilla.org/en-US/docs/Web/API/File>

Riferimenti metodologici

- **E. Gamma, R. Helm, R. Johnson, J. Vlissides**, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- **C4 model for visualising software architecture**:
<https://c4model.com/>
Ultimo accesso: 27 luglio 2026.

2 Tecnologie

2.1 Criteri di scelta

Le tecnologie descritte in questa sezione non sono state selezionate in astratto, sulla base della sola diffusione o popolarità, ma in risposta a bisogni tecnici precisi derivati dal capitolato_G e dall'Analisi dei Requisiti_G. Ciascuno dei bisogni elencati di seguito vincola una o più delle scelte successive, e costituisce il criterio rispetto al quale tali scelte vanno valutate.

1. Risposta progressiva. Un riassunto o una generazione di testo richiedono al modello linguistico un tempo di elaborazione dell'ordine dei secondi. Mantenere l'utente davanti a un semplice indicatore di attesa per tutta la durata dell'operazione non è accettabile sul piano dell'usabilità_G: il testo deve comparire progressivamente mentre viene prodotto (*streaming*). Questo bisogno determina la scelta di un livello server asincrono, di un client HTTP che supporti la lettura incrementale della risposta e di un protocollo di trasporto adatto al flusso unidirezionale di dati testuali. Il criterio discende dal requisito_G **R-109-F-De**, che impone un indicatore di avanzamento per l'intera durata delle operazioni con attesa percepibile, e da **R-79-F-De**, che richiede la possibilità di annullare un'elaborazione in corso: entrambi presuppongono che il server mantenga il controllo dell'operazione mentre questa procede, e non si limiti a restituirne l'esito finale.

2. Riservatezza delle credenziali dei fornitori esterni. L'accesso ai modelli linguistici avviene tramite una chiave di autenticazione fornita dalla proponente_G, in attuazione del requisito di vincolo_G **R-2-V-Ob**, che prescrive l'interfacciamento con i servizi LLM_G tramite API_G. Tale chiave non deve in alcun caso raggiungere il browser, dove sarebbe leggibile da chiunque ispezioni il traffico o il codice dell'applicazione. È quindi necessaria una componente server intermediaria che detenga la chiave e sia l'unico soggetto a comunicare con il fornitore; la medesima componente risolve anche il vincolo_G **R-3-V-Ob** sulla *same-origin policy*, poiché è essa a esporre al browser un'origine controllata, configurata mediante la politica CORS.

Il criterio si estende a ogni fornitore esterno contattato dal prodotto, non al solo fornitore dei modelli. La funzione di generazione a partire da un collegamento (**R-74-F-De**) richiede l'estrazione del contenuto di una pagina web indicata dall'utente, operazione affidata a un secondo servizio esterno dotato di una propria chiave: anche questa è custodita dal backend e non transita dal browser. Va dichiarato, per trasparenza verso l'utente e verso la proponente_G, **quali dati lasciano il perimetro del sistema_G**: verso il fornitore dei modelli linguistici transitano il testo sottoposto all'operazione — la porzione selezionata o l'intera nota — e le istruzioni costruite dall'applicazione; verso il servizio di estrazione transita il solo indirizzo fornito dall'utente. Nessuno dei due riceve dati identificativi dell'utente, che il prodotto non raccoglie.

Il confine è dato da ciò che l'utente sottopone, non dalla provenienza del testo: **nessun file è trasmesso automaticamente**. Una nota aperta da disco resta nel browser finché l'utente non ne affida il contenuto a un'operazione assistita; da quel momento esce dal perimetro come qualunque altro testo. Aprire un file non comporta dunque alcuna trasmissione, chiederne il riassunto sì: la differenza sta in un'azione che l'utente compie deliberatamente.

3. Rendering sicuro del Markdown_G. Il requisito_G **R-113-F-Ob** prevede una componente di rendering che trasformi il testo Markdown_G in una presentazione grafica formattata. Il testo restituito dal modello linguistico non è però contenuto fidato: è generato da un sistema_G esterno a partire da input_G che l'utente controlla solo in parte. Deve essere mostrato all'utente come HTML formattato senza che ciò apra la strada a vulnerabilità di tipo *cross-site scripting* (XSS). La

tecnologia di rendering deve quindi essere sicura per costruzione, non affidarsi a una sanificazione applicata a posteriori. Al medesimo criterio si riconduce **R-110-F-Ob**, che impone messaggi di errore in linguaggio naturale privi di dettagli tecnici: la gestione degli errori del modello non deve riversare nell'interfaccia contenuto proveniente dall'esterno.

4. Editor ricco ma stabile. L'editor deve offrire evidenziazione della sintassi, cronologia delle modifiche con annullamento e ripristino (**R-7-F-De**, **R-8-F-De**), e un comportamento accessibile da tastiera e da lettore di schermo; **R-111-F-Ob** ne prescrive la realizzazione come componente capace di ospitare testo su più righe. Deve inoltre mantenere invariati cursore e posizione di scorrimento mentre il pannello di anteprima si aggiorna in streaming, condizione che esclude soluzioni in cui la ricostruzione della vista coinvolge l'intera area di lavoro.

5. Manutenibilità_G con un gruppo di sette persone. Il prodotto è sviluppato in parallelo da sette persone con livelli di esperienza eterogenei. Ciò richiede codice tipizzato, in cui gli errori di interfaccia emergano in fase di compilazione anziché in esecuzione, confini netti fra i livelli dell'applicazione e verifica_G automatica delle modifiche prima della loro integrazione_G. Il criterio è vincolato da tre requisiti_G di qualità: **R-4-Q-Ob** richiede che ogni pull request_G sia sottoposta a revisione tra pari e superi i test_G automatici prima dell'integrazione_G nel ramo principale, **R-2-Q-Ob** richiede una copertura del codice_G conforme alle soglie del Piano di Qualifica, **R-10-Q-Ob** impone il rispetto delle Norme di Progetto_G. Serve quindi un'infrastruttura di *Continuous Integration_G* che applichi questi controlli in modo automatico e uniforme sui due livelli.

6. Perimetro tecnologico ammesso dal capitolato_G. Il requisito di vincolo_G **R-7-V-Op** stabilisce che la componente server-side, se realizzata, possa essere sviluppata utilizzando **Java** o **Python**. La scelta di Python ricade quindi all'interno del perimetro tecnologico ammesso dalla proponente_G. Sul versante client, **R-1-V-Ob** circoscrive il perimetro alle Tecnologie Web_G e ne fissa gli ambienti di esecuzione di riferimento: le versioni recenti di **Google Chrome**, **Mozilla Firefox** e **Microsoft Edge**. Questo vincolo_G ha conseguenza diretta sulla strategia di persistenza descritta nella sezione 2.4, poiché le tre famiglie di browser non offrono le medesime interfacce di accesso al file system.

Filosofia trasversale. Le scelte che seguono rispondono a un unico criterio di fondo: **l'equilibrio fra controllo e velocità di sviluppo_G**. Il gruppo ha preferito tecnologie che lasciano visibile e modificabile ciò che accade, evitando tanto le soluzioni che nascondono il comportamento dietro astrazioni ampie e opinative, quanto quelle che impongono di riscrivere da zero funzionalità già disponibili. A parità di adeguatezza tecnica, la preferenza è andata agli ecosistemi maturi, con documentazione estesa e ampia disponibilità di risorse: fattore determinante per un gruppo che affronta gran parte di queste tecnologie per la prima volta.

2.2 Frontend

Il frontend_G è un'applicazione a pagina singola eseguita interamente nel browser. Le tecnologie sono presentate nell'ordine con cui si sovrappongono: il linguaggio, la libreria che struttura la vista, l'ecosistema di librerie che ne realizza le funzioni specifiche e, infine, gli strumenti che ne governano la costruzione.

Tabella 1: Tecnologie adottate per il frontend

Tecnologia	Versione	Descrizione e ruolo nel progetto
TypeScript	6.0.3	Linguaggio di sviluppo _G dell'intero frontend _G . Estende JavaScript con un sistema di tipi statici che viene rimosso in fase di compilazione (<i>type erasure</i>): i tipi non esistono a tempo di esecuzione e non introducono alcun costo prestazionale. Il loro contributo è spostare in compilazione errori che si manifesterebbero altrimenti in produzione, in particolare sulle interfacce fra componenti. La validazione dei dati provenienti dall'esterno resta a carico del backend, poiché i tipi non sopravvivono alla compilazione e non possono verificare nulla a tempo di esecuzione.
React	19.2.7	Libreria per la costruzione dell'interfaccia, organizzata a componenti. Editor, anteprima, barra degli strumenti e pannelli delle funzioni assistite dall'LLM _G sono componenti React distinti. Il meccanismo di <i>reconciliation</i> confronta la rappresentazione dell'interfaccia prima e dopo un cambiamento di stato e applica al DOM le sole modifiche effettivamente necessarie: durante lo streaming, in cui lo stato cambia decine di volte al secondo, questo evita la ricostruzione dell'intera pagina. L'applicazione non impiega alcun router : l'interazione si svolge in una sola vista, priva di navigazione fra pagine distinte, e l'introduzione di un instradamento lato client sarebbe una complessità non giustificata.
Zustand	5.0.14	Gestione dello stato applicativo condiviso fra i componenti (testo della nota, selezione corrente, testo in arrivo dallo streaming, stato di errore, modalità di visualizzazione). Espone selettori granulari, ciascuno dei quali osserva una singola porzione di stato: durante lo streaming la ridipintura interessa il solo pannello di anteprima, mentre l'editor resta immobile e conserva cursore e posizione di scorrimento.
react-markdown	10.1.0	Rendering del Markdown _G nel pannello di anteprima. Il testo viene analizzato e trasformato in un albero sintattico astratto, dal quale la libreria costruisce direttamente gli elementi React corrispondenti. In nessun punto della catena viene iniettato HTML grezzo nel documento: la libreria è quindi sicura per costruzione rispetto agli attacchi XSS, senza necessità di un passaggio di sanificazione. Il requisito è critico, poiché il testo da rendere proviene in larga parte dal modello linguistico.
remark-gfm	4.0.1	Estensione della pipeline di analisi con le costruzioni <i>GitHub Flavored Markdown</i> (tabelle, elenchi di attività, testo barrato, collegamenti automatici), attese dall'utenza di riferimento del prodotto. Le tabelle in particolare non appartengono al Markdown _G di base: l'estensione è la condizione perché i requisiti _G R-34-F-De ... R-38-F-De (inserimento di una tabella e gestione di righe e colonne, UC44 ... UC48) trovino una resa nell'anteprima.

Tecnologia	Versione	Descrizione e ruolo nel progetto
remark-math rehype-katex KaTeX	6.0.0 7.0.1 0.17.0	Riconoscimento delle espressioni matematiche nel sorgente <code>Markdown_G</code> e loro composizione tipografica nell'anteprima. Realizzano i requisiti _G R-28-F-De e R-29-F-De (inserimento e rimozione di una formula scientifica o matematica, UC35 e UC37), che senza un motore di composizione dedicato non sarebbero verificabili _G sul lato dell'anteprima. Operano come estensioni della stessa pipeline ad albero sintattico, preservandone la proprietà di non iniettare HTML grezzo.
rehype-highlight highlight.js	7.0.2 11.11.1	Evidenziazione della sintassi all'interno dei blocchi di codice mostrati nell'anteprima. Discende dai requisiti _G R-26-F-De , R-27-F-De e R-119-F-De (inserimento, rimozione e modifica di un blocco di codice sorgente, UC32, UC34 e UC33), che presuppongono una resa del blocco di codice distinta dal testo ordinario.
Tailwind CSS	4.3.1	Sistema di stile <i>utility-first</i> : la presentazione è espressa mediante classi elementari applicate direttamente al marcatore, anziché tramite fogli di stile separati. Ne deriva un insieme chiuso di valori (spaziature, colori, dimensioni tipografiche) che mantiene l'aspetto uniforme fra le parti dell'interfaccia scritte da persone diverse. Il plugin <code>@tailwindcss/typography</code> (0.5.20) fornisce gli stili tipografici applicati al risultato del rendering <code>Markdown_G</code> .
shadcn/ui Radix UI	— 1.5.0	Insieme di componenti di interfaccia (pulsanti, aree di testo, finestre di dialogo, avvisi). <code>shadcn/ui</code> non è una dipendenza installata ma un catalogo da cui il codice sorgente dei componenti viene copiato nel progetto: resta quindi interamente leggibile e modificabile dal gruppo, senza vincolo di adozione verso un fornitore esterno. I componenti poggiano su Radix UI, che fornisce le primitive di comportamento accessibile — navigazione da tastiera, gestione del focus, attributi per i lettori di schermo — la cui realizzazione manuale corretta sarebbe onerosa e soggetta a errori.
CodeMirror 6	6.0.2	Editor di testo del pannello sinistro. Fornisce nativamente evidenziazione della sintassi <code>Markdown_G</code> (modulo <code>@codemirror/lang-markdown</code> 6.5.0), cronologia delle modifiche con annullamento e ripristino (<code>@codemirror/commands</code> 6.10.3) e supporto all'accessibilità, a fronte di un ingombro dell'ordine del centinaio di kilobyte. L'architettura modulare consente di includere le sole estensioni effettivamente utilizzate.
lucide-react	1.18.0	Insieme di icone vettoriali impiegate nella barra degli strumenti e nei comandi dell'interfaccia.
Vite	8.0.16	Strumento di sviluppo _G e costruzione del frontend _G . In sviluppo _G serve i moduli ES nativamente al browser, senza ricostruire l'intero pacchetto a ogni modifica, e aggiorna i moduli modificati mantenendo lo stato dell'applicazione; in produzione genera il pacchetto ottimizzato. Costituisce inoltre la base di configurazione riutilizzata dall'infrastruttura di test descritta nella sezione 2.5.

Alternative considerate

- **Angular, Vue e Svelte al posto di React**

Il confronto documentato nel verbale interno del 12 maggio 2026 ha contrapposto React e Svelte. Svelte adotta un approccio a compilatore: elimina il Virtual DOM spostando in fase di compilazione il lavoro che React svolge in esecuzione, con prestazioni superiori e minore ingombro del pacchetto prodotto. Il suo ecosistema è però più giovane, con minore disponibilità di librerie mature per le esigenze specifiche del prodotto e minore reperibilità di risorse di apprendimento, fattore rilevante per un gruppo che affronta la tecnologia per la prima volta. Vue è a sua volta maturo, ma la via più diffusa per il rendering del Markdown_G nel suo ecosistema consiste nel produrre HTML e iniettarlo nel documento, esattamente il rischio che il criterio 3 impone di evitare. Angular offre un quadro strutturale completo e fortemente prescrittivo, sovradimensionato per un'applicazione a vista singola e con una curva di apprendimento incompatibile con i tempi del progetto. La determinante è che le librerie richieste — rendering Markdown_G sicuro, componenti accessibili, integrazione_G dell'editor — individuano React come primo ambiente di destinazione.

- **Redux, Context API e Jotai al posto di Zustand**

Redux impone una quantità di codice di supporto (azioni, riduttori, selettori memorizzati) sproporzionata rispetto alla dimensione dello stato da gestire. La Context API, disponibile nativamente in React, non offre sottoscrizioni granulari: ogni variazione del valore del contesto provoca la ridipintura di tutti i componenti che vi accedono, comportamento penalizzante durante lo streaming, in cui lo stato cambia con frequenza elevata e ridipingere l'editor comporterebbe la perdita della posizione del cursore. Jotai adotta un modello ad atomi tecnicamente adeguato, ma richiede di scomporre lo stato in unità elementari fin dall'inizio: Zustand ottiene il medesimo risultato con un modello concettuale più semplice e una superficie di apprendimento minore.

- **marked e markdown-it al posto di react-markdown**

Entrambe le librerie convertono il Markdown_G in una stringa HTML, che dovrebbe poi essere inserita nel documento tramite un meccanismo di iniezione diretta. Ciò renderebbe obbligatorio un passaggio esplicito di sanificazione, la cui correttezza dipenderebbe dalla configurazione adottata e andrebbe verificata_G separatamente. react-markdown rende il problema inesistente per costruzione.

- **MUI e Chakra UI al posto di shadcn/ui**

Offrono un numero maggiore di componenti già pronti, ma impongono un linguaggio visivo marcato, la cui personalizzazione richiede di intervenire contro le impostazioni predefinite della libreria; inoltre il codice dei componenti resta all'interno della dipendenza, non modificabile. L'alternativa opposta, i CSS Modules, non aggiunge alcuna dipendenza ma non fornisce né un sistema di valori condiviso né componenti accessibili, che andrebbero realizzati integralmente dal gruppo.

- **Monaco e textarea nativa al posto di CodeMirror**

Monaco è l'editor impiegato in Visual Studio Code: è progettato per la scrittura di codice sorgente e la sua dotazione di funzioni — completamento automatico, analisi del linguaggio, riquadro di anteprima — eccede quanto richiesto dalla redazione di note, a fronte di un ingombro di oltre un ordine di grandezza superiore. L'elemento `textarea` nativo non comporta alcuna dipendenza, ma è privo di evidenziazione della sintassi e di una cronologia delle modifiche strutturata.

- **Webpack e Create React App al posto di Vite**

Ricostruiscono l'intero pacchetto a ogni modifica, con tempi di attesa in sviluppo_G sensibilmente superiori, e richiedono una configurazione più estesa e frammentata. Create React App non è

inoltre più oggetto di manutenzione attiva.

2.3 Backend

Il backend è una componente server che espone alcune API_G tramite HTTP. Le sue responsabilità sono quattro:

1. custodire le chiavi di accesso ai fornitori esterni, così che non raggiungano mai il browser (criterio 2);
2. comporre le istruzioni da inviare al modello linguistico;
3. ritrasmettere al browser il testo prodotto man mano che diviene disponibile (criterio 1);
4. acquisire il contenuto testuale di una pagina web indicata dall'utente, quando la generazione avviene a partire da un collegamento.

La quarta responsabilità discende dal requisito_G **R-74-F-De** (UC67.2 e UC67.5) e non è riducibile alle altre tre: comporta un secondo fornitore esterno, distinto da quello dei modelli, e per questo è trattata separatamente nel paragrafo *Estrazione del contenuto delle pagine web*. Come per il frontend_G, le tecnologie sono presentate nell'ordine linguaggio, framework_G, librerie, strumenti.

Tabella 2: Tecnologie adottate per il backend

Tecnologia	Versione	Descrizione e ruolo nel progetto
Python	3.12	Linguaggio di sviluppo _G del backend. Rientra nel perimetro tecnologico ammesso dal requisito _G R-7-V-Op. Dispone del supporto nativo alla programmazione asincrona (async/await), condizione necessaria per gestire attese di rete prolungate senza bloccare il processo, e dell'ecosistema più esteso per l'integrazione _G con i modelli linguistici.
FastAPI	0.136.1	Framework _G web su cui è costruita l'API _G . È asincrono per impostazione, requisito imprescindibile per lo streaming; deriva la validazione dei corpi delle richieste direttamente dalle annotazioni di tipo, senza codice di controllo scritto a mano; genera automaticamente lo schema OpenAPI dell'interfaccia esposta. La risposta in streaming è realizzata mediante StreamingResponse , che consuma un generatore asincrono e gestisce la trasmissione a blocchi lungo la connessione HTTP.
Pydantic	2.13.4	Validazione e serializzazione dei dati in ingresso e in uscita. Gli schemi Pydantic costituiscono la fonte unica di verità dei contratti dell'API _G : da essi FastAPI deriva lo schema OpenAPI e, da quest'ultimo, vengono generati i tipi TypeScript utilizzati dal frontend _G (si veda la sezione 2.6). Il contratto è definito quindi una volta sola, in un solo punto del sistema _G .
pydantic-settings	2.14.1	Lettura e validazione della configurazione applicativa (indirizzo del gateway dei modelli, identificativo del modello, chiave di accesso, origini ammesse dalla politica CORS) a partire dalle variabili d'ambiente. Le impostazioni sono esposte tramite un'unica istanza per processo.

Tecnologia	Versione	Descrizione e ruolo nel progetto
httpx	0.28.1	Client HTTP asincrono impiegato per la comunicazione verso il provider dei modelli linguistici. Supporta la lettura della risposta a blocchi man mano che arriva, senza attenderne il completamento, e propaga l'annullamento lungo l'intera catena: se l'utente interrompe la generazione, la chiusura si trasmette dal browser al backend e da questo alla connessione verso il provider, senza che rimangano richieste attive prive di destinatario.
tavily-python	0.7.26	Estrazione del contenuto testuale delle pagine web nella funzione di generazione a partire da un collegamento (requisito _G R-74-F-De , UC67.2 e UC67.5). Restituisce il solo testo della pagina, già ripulito dal marcatore e dagli elementi accessori, e ne consente il troncamento a una lunghezza massima prima dell'invio al modello. È un client verso un servizio esterno: si veda il paragrafo <i>Estrazione del contenuto delle pagine web</i> per il perimetro dei dati trasmessi e per l'alternativa scartata.
Uvicorn	0.47.0	Server ASGI sul quale l'applicazione FastAPI è posta in esecuzione. Costituisce l'implementazione dell'interfaccia asincrona fra il server HTTP e il codice applicativo.
Server-Sent Events	—	Protocollo di trasporto dei blocchi testuali dal backend al browser. È un formato testuale unidirezionale che viaggia su una normale connessione HTTP mantenuta aperta, in cui ogni evento è una riga con prefisso data: e la fine del flusso è segnalata da un marcatore convenzionale. La direzione unica — dal server al client — corrisponde esattamente alla natura del problema.

Accesso ai modelli linguistici. L'accesso ai modelli avviene attraverso un **gateway LiteLLM**, componente esterna che espone un'interfaccia compatibile con quella di OpenAI e uniforma l'accesso a fornitori diversi. Poiché si tratta di un servizio contattato via HTTP e non di una libreria installata nel progetto, non figura fra le dipendenze del backend e non è associato a una versione dichiarata nei file di progetto.

La comunicazione con il gateway è confinata dietro l'astrazione **LLMClient**, una classe astratta che dichiara la sola operazione di produzione incrementale del testo. L'implementazione concreta che dialoga con LiteLLM — interpretandone il formato di risposta e traducendone gli errori — ne è l'unico realizzatore, secondo il pattern Adapter descritto nella sezione dedicata all'architettura. **Né i router né la logica di dominio_G invocano direttamente LiteLLM:** conoscono unicamente l'interfaccia astratta. La sostituzione del gateway con un fornitore diverso comporta pertanto la scrittura di una nuova implementazione dell'interfaccia, e nessuna modifica al resto del backend.

Estrazione del contenuto delle pagine web. Il requisito_G **R-74-F-De** consente all'utente di indicare un URL come fonte di contesto per la generazione automatica di testo (UC67.2). Poiché il modello linguistico non accede alla rete, il contenuto della pagina deve essere acquisito dal prodotto e passato al modello come parte delle istruzioni. L'operazione è affidata a **Tavily**, servizio esterno che restituisce il testo di una pagina già ripulito dal marcatore, dalla navigazione e dagli elementi accessori.

Si tratta del **secondo fornitore esterno** del sistema_G, dotato di una propria chiave di accesso. Coerentemente con il criterio 2, la chiave è custodita dal backend e non raggiunge il browser; verso il servizio transita esclusivamente l'indirizzo indicato dall'utente, non il testo della

nota. Il contenuto restituito è troncato a una lunghezza massima prima di essere inoltrato al modello, così da mantenere prevedibile la dimensione delle istruzioni.

L'alternativa considerata è l'**estrazione lato server con una libreria locale**: *trafilatura* e le realizzazioni Python dell'algoritmo *Readability* scaricano la pagina con il client HTTP già presente nel progetto e ne isolano il contenuto principale senza coinvolgere alcun terzo. Eliminerebbero un fornitore, una chiave e un punto di uscita dei dati — vantaggio non trascurabile — ma sposterebbero sul gruppo la qualità dell'estrazione, che su pagine costruite dinamicamente tramite JavaScript risulta sensibilmente inferiore, e richiederebbero di gestire in proprio reindirizzamenti, codifiche e tempi di risposta dei siti contattati. Poiché R-74-F-De è desiderabile e non obbligatorio, il gruppo ha preferito la soluzione che ne rende prevedibile il completamento, accettando la dipendenza esterna.

Rispetto all'accesso ai modelli linguistici **non sussiste alcuna asimmetria**: le due dipendenze esterne sono trattate allo stesso modo. Come l'accesso al gateway passa dall'interfaccia astratta `LLMClient`, così l'estrazione passa dall'interfaccia astratta `ContentExtractor`, dichiarata anch'essa fra le porte del dominio_G e affiancata dal proprio tipo di errore, così che chi la consuma possa trattarne il fallimento senza conoscere l'implementazione che lo solleva. Ciascuna delle due porte ha un unico realizzatore concreto fra gli adattatori verso l'esterno, ciascuna è costruita da un provider dedicato nel medesimo modulo di composizione, e ciascuna raggiunge la rotta che se ne serve per iniezione della dipendenza: la rotta di generazione a partire da un collegamento dichiara entrambe le porte fra i propri parametri e non nomina alcun fornitore.

La conseguenza è la stessa rivendicata per il gateway: **sostituire il servizio di estrazione comporta la scrittura di un nuovo adattatore e la modifica del solo modulo di composizione**, non l'intervento sul dominio_G o sulle rotte. La differenza fra le due dipendenze non riguarda perciò la loro sostituibilità, ma il perimetro dei dati e la criticità: l'assenza della chiave del gateway impedisce l'avvio del processo, poiché senza modello linguistico nessuna funzione assistita è servibile, mentre l'assenza della chiave del servizio di estrazione degrada la sola generazione a partire da un collegamento e viene segnalata per singola richiesta.

Alternative considerate

• Node.js con TypeScript e Go al posto di Python

Entrambi sarebbero tecnicamente adeguati: Node.js consentirebbe di adottare un unico linguaggio sui due livelli, Go offrirebbe prestazioni superiori nella gestione di connessioni concorrenti prolungate. Nessuno dei due rientra però nel perimetro tecnologico ammesso dal requisito_G R-7-V-Op, che indica Java e Python. Java rientra nel perimetro ed è stato valutato, ma è stato scartato per la minore immediatezza del suo ecosistema nell'integrazione_G con i modelli linguistici e per la maggiore verbosità rispetto alla dimensione contenuta del backend, che espone poche interfacce e non contiene logica di dominio_G estesa.

• Flask e Django al posto di FastAPI

Flask è sincrono per impostazione: il supporto allo streaming asincrono richiederebbe estensioni aggiuntive e una configurazione non prevista dal suo modello di base, oltre a un livello di validazione scritto separatamente. Django è orientato ad applicazioni complete, con mappatore relazionale a oggetti, sistema di autenticazione e pannello di amministrazione integrati: componenti che il prodotto non utilizza, dato che non prevede persistenza su base di dati (si veda la sezione 2.4), e che ne renderebbero l'adozione sproporzionata.

• requests e aiohttp al posto di httpx

La libreria `requests` è lo standard di fatto per le richieste HTTP in Python, ma è esclusivamente sincrona: bloccherebbe il processo per l'intera durata della risposta del modello, vanificando la

scelta di un framework_G asincrono. La libreria `aiohttp` è asincrona e supporta lo streaming, ma espone un'ergonomia meno lineare nella gestione della chiusura anticipata delle connessioni, aspetto centrale per la funzione di annullamento della generazione.

- **SDK diretto del fornitore e LangChain al posto del gateway**

L'impiego dell'SDK proprietario di un singolo fornitore legherebbe il codice a quel fornitore, in un ambito in cui modelli e condizioni di accesso cambiano con frequenza elevata. LangChain, all'estremo opposto, introduce un livello di astrazione ampio e fortemente opinato — catene, agenti, memoria, strumenti — di cui il prodotto non utilizzerebbe che una frazione minima, a fronte di un aumento sensibile della superficie di dipendenza e di una minore prevedibilità del comportamento. Il gateway, unito all'interfaccia `LLMClient`, offre l'indipendenza dal fornitore senza alcuno dei due inconvenienti.

- **WebSocket e polling al posto dei Server-Sent Events**

I WebSocket stabiliscono un canale bidirezionale, con un protocollo di negoziazione dedicato e la necessità di gestire esplicitamente riconessioni e stato della connessione: capacità superflue, poiché nel prodotto i dati scorrono in una sola direzione. Il *polling* periodico richiederebbe di accumulare il testo prodotto lato server in attesa di essere richiesto, introducendo latenza e complessità di gestione dello stato intermedio.

2.4 Persistenza delle note

Il gruppo ha deliberato di non realizzare la persistenza delle note su base di dati. Le note non lasciano dunque la macchina dell'utente: sono salvate e ricaricate sul **file system locale** per il tramite del browser, e conservate fra una sessione e l'altra nell'**archivio locale del browser** stesso. Entrambi i meccanismi sono lato client; nessun dato dell'utente raggiunge il servizio applicativo per esservi memorizzato.

Meccanismo adottato. I meccanismi di persistenza lato client sono **due, distinti per scopo e non alternativi**. L'accesso al file system serve all'**esportazione e all'importazione deliberate**: è l'utente a chiedere esplicitamente di salvare una nota su disco o di aprirne una, e il file prodotto è un documento che gli appartiene e che può usare con altri strumenti. La persistenza nel browser serve invece alla **continuità della sessione**: senza di essa chiudere la scheda comporterebbe la perdita delle note in lavorazione, poiché non esiste alcun archivio server-side che le conservi. Il primo meccanismo è descritto di seguito, il secondo nel paragrafo *Continuità della sessione*.

Le operazioni di salvataggio e di caricamento su file sono realizzate con una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Quando il browser espone la **File System Access API** — verificata mediante la presenza dei metodi `showOpenFilePicker` e `showSaveFilePicker` sull'oggetto globale della finestra — l'applicazione apre la finestra di selezione file nativa del sistema_G operativo. Il caricamento ottiene un riferimento al file scelto e ne legge il contenuto testuale; il salvataggio ottiene un riferimento al percorso di destinazione e vi scrive un flusso di byte. L'annullamento da parte dell'utente è riconosciuto e trattato come esito legittimo, non come errore.

Quando l'interfaccia non è disponibile — condizione che ricorre su **Firefox** e **Safari**, che non la implementano — l'applicazione ricade su meccanismi supportati universalmente. La doppia via non è una precauzione generica: il requisito di vincolo_G **R-1-V-Ob** include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API lascerebbe scoperto uno dei tre browser prescritti. In caricamento crea un elemento `input` di tipo `file` e ne legge il contenuto tramite un lettore di file; in salvataggio costruisce un

oggetto `Blob` con il contenuto della nota, ne ricava un indirizzo temporaneo e lo assegna a un collegamento di scaricamento attivato per via programmatica, delegando al browser la scelta della destinazione secondo le impostazioni dell'utente. L'indirizzo temporaneo viene rilasciato al termine dell'operazione.

Il criterio di selezione è quindi la disponibilità effettiva dell'interfaccia nel browser in uso, non l'identificazione del browser stesso: l'applicazione non interroga l'agente utente, ma verifica direttamente la presenza dei metodi richiesti. La funzione resta pertanto disponibile su ogni browser, con la sola differenza che sui browser dotati dell'interfaccia nativa l'utente sceglie esplicitamente il percorso di destinazione, mentre sugli altri il file segue le impostazioni di scaricamento predefinite.

Continuità della sessione. Il secondo meccanismo non richiede alcuna azione dell'utente. Lo store delle note è avvolto in un middleware di persistenza che ne riversa lo stato nel `localStorage` del browser sotto la chiave `notes_persistence`, e lo rilegge all'avvio dell'applicazione: alla riapertura l'elenco delle note e la nota corrente sono quelli con cui la sessione precedente si era chiusa, e il contenuto della nota corrente è ricaricato nell'editor. Ciò che viene conservato è l'elenco delle note con i rispettivi identificativo, titolo, contenuto e istanti di creazione e di ultima modifica, oltre all'indicazione di quale sia la nota corrente.

Va precisato **quando** il testo in lavorazione raggiunge quell'archivio, per non attribuire al meccanismo una copertura che non ha: il contenuto dell'editor è riversato nella nota corrente in occasione del cambio di nota e della creazione di una nuova nota, non a ogni modifica del testo né alla chiusura della scheda. I metadati, assegnati alla creazione, sono invece persistiti immediatamente. Le modifiche successive all'ultimo cambio di nota non sono quindi conservate: è un limite noto della realizzazione attuale, non una proprietà del supporto.

I limiti del supporto sono quelli propri dell'archiviazione locale del browser e vanno dichiarati. Lo spazio disponibile è soggetto a una **quota**, il cui superamento fa fallire la scrittura: l'applicazione lo tratta come condizione prevista e ripristina lo stato precedente anziché lasciare l'interfaccia in uno stato incoerente. I dati sono **confinati all'origine e al profilo del browser** in uso: non sono raggiungibili da un altro browser, da un altro dispositivo o da una finestra di navigazione anonima, non vi è alcuna sincronizzazione fra dispositivi, e la cancellazione dei dati di navigazione li rimuove. Questo meccanismo **non è un sostituto della persistenza server-side**: resta interamente lato client, e non copre alcuno dei requisiti_G opzionali elencati più avanti in questa sezione.

Formato dei file. Le note sono salvate in `MarkdownG`, con estensione `.md` e tipo MIME `text/markdown`; in caricamento sono accettate anche le estensioni `.txt`. La scelta del testo semplice attua il requisito di vincolo_G **R-4-V-Ob**, che prescrive la memorizzazione delle note salvate localmente come file di testo. Il file contiene il solo testo della nota, senza intestazioni o marcatori aggiuntivi: resta quindi leggibile e modificabile con qualsiasi altro strumento, e non introduce alcun vincolo_G verso il prodotto.

Metadati della nota. I metadati che l'applicazione associa alla nota — identificativo, titolo, istante di creazione e istante di ultima modifica — non sono scritti nel file, che resta un documento `MarkdownG` puro. Il titolo è veicolato dal nome del file: in salvataggio il nome proposto deriva dal titolo della nota, in caricamento il titolo è ricavato dal nome del file privato dell'estensione. La derivazione avviene in **entrambi** i rami di caricamento, quello nativo e quello di ripiego, con un titolo convenzionale riservato al solo caso in cui il file scelto non esponga un nome utilizzabile: la parità fra i due rami è necessaria, poiché quello di ripiego è il percorso primario su Firefox, incluso fra i browser prescritti dal requisito_G di vincolo_G **R-1-V-Ob**.

Sul requisito_G **R-97-F-De** (UC84), che richiede la visualizzazione delle informazioni e dei metadati di una nota selezionata, vanno distinti **due casi**, che non hanno la medesima copertura.

- **Nota creata nel prodotto.** Identificativo, titolo e istanti di creazione e di ultima modifica sono assegnati alla creazione e affidati alla persistenza locale descritta sopra, che li scrive immediatamente: **sopravvivono alla chiusura della sessione** e sono disponibili alla riapertura, alle condizioni proprie di quel supporto (stessa origine, stesso profilo del browser). Per queste note il requisito_G è coperto dai dati che l'applicazione già possiede, senza che nulla debba essere ricavato dal file system.
- **Nota importata da disco.** Il file contiene il solo testo, quindi i metadati vanno ricostruiti da ciò che il browser espone sull'oggetto **File**. Il titolo lo è già, come appena descritto; gli istanti no: all'importazione entrambi sono posti all'istante di apertura, che descrive quando la nota è entrata nell'applicazione e non quando il file è stato scritto o modificato.

Trattandosi di un requisito_G **desiderabile** e non opzionale, il secondo caso non può essere rinviato.

Per le note importate il recupero è ottenuto senza modificare il formato del file, sfruttando i metadati che il file system già espone. L'oggetto **File** restituito sia dalla File System Access API sia dall'elemento **input** di tipo file riporta gli attributi **name**, **lastModified** e **size**: all'apertura di una nota l'applicazione li impiega per popolare rispettivamente titolo, istante di ultima modifica e dimensione del contenuto. Il numero di caratteri e il numero di parole sono derivati dal testo caricato. Resta non ricostruibile il solo **istante di creazione**, che il file system espone in modo non uniforme fra le piattaforme e che le interfacce del browser non rendono disponibile: per le sole note importate da disco tale informazione è presentata come non disponibile anziché con un valore convenzionale, così che l'interfaccia non affermi un dato che il prodotto non possiede. Per le note create nel prodotto l'istante di creazione è invece un dato posseduto e conservato, e va presentato come tale.

Motivazione dell'esclusione della base di dati. La persistenza server-side è prevista dal capitolato_G come estensione facoltativa. I requisiti_G funzionali che ne derivano sono classificati come opzionali nell'Analisi dei Requisiti_G e riguardano quattro capacità distinte:

- **operazioni sull'archivio remoto:** R-83-F-Op (creazione di una nota nella base di dati, UC74.2), R-86-F-Op (caricamento, UC76.2), R-89-F-Op (salvataggio, UC78.2), R-92-F-Op (rimozione, UC80.2), R-95-F-Op (rinomina, UC82.2). Sono il nucleo di ciò che l'esclusione comporta: le quattro operazioni di base sulle note, più la rinomina;
- **ricerca e filtro:** R-98-F-Op (ricerca per titolo), R-99-F-Op (ricerca full-text), R-100-F-Op e R-101-F-Op (filtro per data di creazione e di ultima modifica);
- **autenticazione:** R-102-F-Op (accesso con credenziali), R-103-F-Op (gestione degli errori di autenticazione), R-108-F-Op (disconnessione);
- **gestione dell'account:** R-104-F-Op (registrazione), R-105-F-Op (unicità dello username), R-106-F-Op (conferma della password), R-107-F-Op (rimozione dell'account e dei dati associati).

Sono opzionali anche i requisiti_G di vincolo_G corrispondenti: **R-5-V-Op** (memorizzazione delle note in una base di dati raggiungibile dal sistema_G), **R-6-V-Op** (componente server raggiungibile tramite chiamate HTTP) e **R-7-V-Op** (linguaggio della componente server-side). Si segnala che dei tre soltanto R-7-V-Op è già soddisfatto dall'architettura attuale, poiché la componente

server esiste per le funzioni assistite da LLM_G indipendentemente dalla persistenza; R-5-V-Op e R-6-V-Op restano scoperti insieme alla funzionalità che li condiziona.

Il gruppo ha scelto di concentrare le risorse della presente iterazione sul completamento dei requisiti_G obbligatori e desiderabili, in particolare sulle funzionalità assistite da LLM_G , che costituiscono il nucleo del prodotto richiesto dalla proponente_G. Lo stato di copertura di ciascun requisito_G è riportato nella sezione 5.

Va precisato l'effettivo margine di estensione, per non attribuire all'architettura una predisposizione che non possiede. Nel perimetro attuale la persistenza delle note risiede **interamente nel frontend_G**: le note non raggiungono in alcun momento il backend, e nel dominio_G del backend non esiste una porta per le note in attesa di essere implementata. Introdurre la persistenza su base di dati comporterebbe quindi lavoro nuovo e non marginale: entità di dominio_G per la nota e per l'utente, le porte corrispondenti, gli endpoint che le espongono, un adattatore verso il sistema_G di gestione della base di dati e l'intero blocco di autenticazione richiesto da R-102-F-Op ... R-108-F-Op.

Ciò che l'architettura esagonale garantisce è un'altra proprietà, più circoscritta ma sufficiente: la persistenza sarebbe aggiunta **come nuova porta e nuovo adattatore**, senza perturbare il dominio_G delle funzioni assistite da LLM_G già realizzato. Il costo dell'estensione ricade sulle componenti nuove, non sulla riscrittura di quelle esistenti.

2.5 Tecnologie di test e analisi

Le attività di verifica_G previste dalle Norme di Progetto_G si articolano in analisi statica_G, condotta sul codice sorgente senza porlo in esecuzione, e analisi dinamica_G, condotta eseguendo il codice su casi di prova predisposti.

2.5.1 Analisi statica

L'analisi statica_G intercetta gli errori prima che il codice venga eseguito, spostandone il rilevamento dalla fase di prova a quella di scrittura.

Tabella 3: Strumenti di analisi statica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Compilatore TypeScript	6.0.3	Verifica _G della coerenza dei tipi sull'intero codice del frontend _G . La compilazione è eseguita in modo esplicito come primo passo del comando di costruzione (<code>tsc -b && vite build</code>): un errore di tipo interrompe la costruzione e impedisce la produzione del pacchetto. La configurazione attiva inoltre la segnalazione di variabili e parametri dichiarati e non utilizzati, dei casi non terminati nei costrutti di selezione multipla e delle costruzioni non rimovibili per sola cancellazione dei tipi.
ESLint	10.5.0	Analisi statica _G del codice del frontend _G rispetto a un insieme di regole configurate. Individua costrutti sintatticamente validi ma problematici, che il compilatore non segnala.
typescript-eslint	8.61.0	Estensione di ESLint alla sintassi TypeScript e alle regole specifiche del linguaggio.

Tecnologia	Versione	Descrizione e ruolo nel progetto
eslint-plugin-react-hooks	7.1.1	Verifica _G del rispetto delle regole di impiego degli <i>hook</i> di React, in particolare della completezza degli elenchi di dipendenze. Si tratta di una classe di errori che non produce alcun messaggio in esecuzione ma si manifesta come stato dell'interfaccia non aggiornato, di difficile diagnosi.
eslint-plugin-react-refresh	0.5.2	Verifica _G che i moduli rispettino le condizioni necessarie all'aggiornamento a caldo dei componenti durante lo sviluppo _G .
Ruff	0.16.1	Analisi statica _G e formattazione automatica del codice del backend. Riunisce in un solo strumento le regole di più analizzatori dell'ecosistema Python: gli insiemi selezionati sono quelli di pyflakes , pycodestyle (errori e avvisi), isort , pyupgrade e flake8-bugbear , che segnalano importazioni e variabili non utilizzate, ordinamento delle importazioni, costrutti sintatticamente validi ma problematici e violazioni delle convenzioni di stile. Il formattatore integrato riproduce lo stile di black , indicato dalle Norme di Progetto _G per la formattazione del codice Python: la conformità è quindi mantenuta senza introdurre un secondo strumento. La configurazione risiede in ruff.toml , accanto ai file di progetto del backend.
mypy	—	Verifica _G statica delle annotazioni di tipo del backend, in modalità rigorosa. Controlla la coerenza fra le firme dichiarate e le chiamate effettive, la completezza delle annotazioni e la correttezza dei tipi dei generatori asincroni impiegati nello streaming — classe di errori che, in un linguaggio a tipizzazione dinamica, si manifesterebbe altrimenti solo in esecuzione e, nel caso dello streaming, a operazione già avviata. Porta il backend al medesimo livello di controllo che il compilatore TypeScript garantisce sul frontend _G .

L'analisi statica_G è quindi applicata a **entrambi** i livelli del prodotto, come richiesto dalle Norme di Progetto_G e, per il loro tramite, dal requisito_G di qualità **R-10-Q-Ob**. La simmetria fra i due livelli è voluta: al compilatore TypeScript ed ESLint sul frontend_G corrispondono mypy e Ruff sul backend, con la medesima ripartizione fra verifica_G dei tipi e verifica_G delle regole di scrittura.

Si osserva che la validazione operata dagli schemi **Pydantic** non appartiene a questa categoria e non ne costituisce un sostituto: Pydantic controlla i *dati* che attraversano i confini dell'applicazione a tempo di esecuzione, mentre l'analisi statica_G controlla il *codice* senza eseguirlo. Le due verifiche_G intercettano difetti disgiunti — un corpo di richiesta malformato nel primo caso, un'incoerenza fra firme di funzione nel secondo — e sono pertanto complementari.

2.5.2 Analisi dinamica

L'analisi dinamica_G è organizzata su test_G di unità e di integrazione_G dei due livelli, eseguiti separatamente dalle rispettive catene di strumenti.

Tabella 4: Strumenti di analisi dinamica

Tecnologia	Versione	Descrizione e ruolo nel progetto
Vitest	4.1.8	Esecutore dei test _G di unità e di integrazione _G del frontend _G . Riutilizza integralmente la configurazione di Vite — risoluzione dei moduli, alias dei percorsi, trasformazione di TypeScript e JSX — evitando la manutenzione di una seconda catena di strumenti allineata alla prima.
@vitest/coverage-v8	4.1.8	Misurazione della copertura del codice _G raggiunta dai test _G del frontend _G . Si appoggia allo strumento di copertura nativo del motore V8, senza richiedere una strumentazione del codice sorgente. Le soglie minime sono dichiarate nella configurazione di Vitest, così che il superamento sia verificato _G dall'esecutore stesso e non affidato alla lettura del rapporto; l'insieme dei file misurati vi è dichiarato esplicitamente, così che un sorgente privo di test pesi sul denominatore anziché sparire dal calcolo. È la controparte di pytest-cov sul versante frontend _G : senza di essa il Piano di Qualifica non potrebbe riportare una metrica _G di copertura su questo livello, come richiesto da R-2-Q-Ob .
jsdom	29.1.1	Realizzazione del DOM in ambiente Node.js, che consente di eseguire i test _G dei componenti senza avviare un browser.
Testing Library (React)	16.3.2	Interrogazione dei componenti resi attraverso ruoli, etichette e testo visibile, così come vi accederebbe una persona che utilizza il prodotto, anziché attraverso la struttura interna del marcatore. I test _G restano quindi validi a fronte di riorganizzazioni interne dei componenti e verificano indirettamente l'accessibilità dell'interfaccia.
jest-dom user-event	6.9.1 14.6.1	Asserzioni specifiche per gli elementi del DOM e simulazione delle interazioni dell'utente (digitazione, selezione, attivazione di comandi) con la sequenza di eventi effettivamente prodotta da un browser.
pytest	9.0.3	Esecutore dei test _G del backend. Il sistema delle <i>fixture</i> consente di comporre e riutilizzare le precondizioni dei test _G in modo esplicito.
pytest-asyncio	1.4.0	Esecuzione dei test _G su funzioni asincrone, condizione necessaria data la natura asincrona dell'intero backend. La modalità automatica configurata nel progetto rende superflua l'annotazione di ogni singolo test _G .
pytest-cov	7.1.0	Misurazione della copertura del codice _G raggiunta dai test _G del backend.
respx	0.23.1	Interposizione sulle richieste HTTP effettuate tramite httpx. Consente di verificare _G il comportamento del client verso il provider dei modelli — flusso corretto, risposta malformata, errore di connessione, scadenza del tempo massimo — senza contattare alcun servizio esterno, rendendo i test _G deterministici e indipendenti dalla rete.

Alternative considerate

- **Jest al posto di Vitest**

Jest è lo standard storico dei test_G in ambiente JavaScript, con la comunità più ampia e la

maggiore disponibilità di documentazione. Richiederebbe però una configurazione separata e mantenuta in parallelo a quella di Vite, in particolare per la trasformazione di TypeScript e per i moduli ES, che Jest non supporta nativamente. Il disallineamento fra le due configurazioni è una fonte di errori ricorrente, e non è compensato da alcun vantaggio funzionale: l'interfaccia di programmazione di Vitest è deliberatamente compatibile con quella di Jest, e i `testG` risultano pressoché identici.

- **unittest al posto di pytest**

Il modulo `unittest` è incluso nella libreria standard di Python e non aggiunge dipendenze. Impone però una struttura a classi che rende i `testG` più verbosi e, soprattutto, non dispone né di un sistema di *fixture* componibili paragonabile a quello di `pytest` né dell'ecosistema di estensioni sul quale il progetto si appoggia per l'esecuzione asincrona, la misurazione della copertura del codice_G e l'interposizione sulle richieste HTTP.

2.6 Infrastruttura

Le tecnologie di questa sezione non concorrono al comportamento del prodotto in esecuzione, ma governano il modo in cui esso viene costruito, avviato, verificato_G e mantenuto coerente fra i due livelli.

Organizzazione della repository_G. Prima delle tecnologie va richiamata una decisione organizzativa che le condiziona tutte: il gruppo ha adottato un'unica repository_G (*mono-repo*) per i due livelli del prodotto. Il codice del frontend_G risiede in `/web` e quello del backend in `/api`, ciascuno con il proprio gestore di pacchetti, il proprio file di dipendenze e il proprio file di lock. Restano quindi due progetti distinti, con costruzioni e dipendenze proprie: condividono la repository_G e la cronologia delle modifiche, non divengono un unico artefatto_G.

La scelta ha due conseguenze dirette sulle tecnologie descritte di seguito. La prima è che una variazione del contratto dell'API_G e il corrispondente adeguamento del client possono essere integrati_G con una sola pull request_G, che li sottopone alla medesima verifica_G automatica: non esiste un intervallo in cui i due livelli risultino disallineati. La seconda è che la configurazione di verifica_G è definita una volta sola, in un solo insieme di flussi di lavoro.

Tabella 5: Tecnologie di infrastruttura

Tecnologia	Versione	Descrizione e ruolo nel progetto
Docker Docker Compose	29.4.3 5.1.4	Ambiente di sviluppo _G ed esecuzione riproducibile. Il file <code>docker-compose.yml</code> descrive i due servizi <code>web</code> e <code>api</code> , la rete che li collega, le porte esposte verso la macchina ospite e le variabili d'ambiente, e li avvia con un solo comando. Le immagini di base sono fissate dai rispettivi <code>Dockerfile</code> : <code>node:22-alpine</code> per il frontend _G e <code>python:3.12-slim</code> per il backend. La linea di <code>Node.js</code> non è scelta soltanto come limite inferiore: è fissata al di sopra e al di sotto dal campo <code>engines</code> di <code>package.json</code> (<code>>=22 <23</code>) e dal file <code>.nvmrc</code> , che l'ambiente locale e l'integrazione _G continua leggono entrambi. Il limite superiore ha una ragione misurata e documentata nel repository _G : su linee successive parte della suite di test _G del frontend _G non è eseguibile.

Tecnologia	Versione	Descrizione e ruolo nel progetto
GitHub Actions	—	Infrastruttura di <i>Continuous Integration</i> _G . Su ogni pull request _G , e su ogni integrazione _G nei rami principali, esegue in processi indipendenti l'analisi statica _G dei due livelli, la verifica _G dei contratti di dipendenza del backend, il controllo che il contratto dell'API _G e i tipi da esso derivati siano allineati al codice, le due suite di test _G con misurazione della copertura del codice _G e la costruzione del pacchetto di produzione del frontend _G . È il luogo in cui gli strumenti delle sezioni 2.5.1 e 2.5.2 vengono effettivamente applicati: senza di esso la loro esecuzione dipenderebbe dalla diligenza individuale. Trattandosi di un servizio della piattaforma e non di una dipendenza installata, non è associato a una versione dichiarata nei file di progetto; le versioni fissate sono quelle delle singole azioni richiamate dai flussi di lavoro.
pnpm	11.18.0	Gestore dei pacchetti del frontend _G . Memorizza ogni versione di ogni pacchetto una sola volta e la collega per riferimento, riducendo occupazione di spazio e tempi di installazione. Il file <code>pnpm-lock.yaml</code> fissa le versioni risolte dell'intero albero delle dipendenze; la versione dello strumento stesso è vincolata dal campo <code>packageManager</code> di <code>package.json</code> , che l'integrazione _G continua legge per approvvigionarsi, così che questa e lo sviluppo _G locale adoperino il medesimo risolutore. Il vincolo _G non si estende all'immagine di sviluppo _G del frontend _G , che installa lo strumento senza indicarne la versione: chi lavora nel container può quindi risolvere una versione diversa da quella dichiarata.
uv	0.11.16	Gestore dei pacchetti e dell'ambiente virtuale del backend. Risolve le dipendenze dichiarate in <code>pyproject.toml</code> e ne fissa le versioni in <code>uv.lock</code> ; l'installazione in fase di costruzione dell'immagine avviene in modalità vincolata al file di lock, così che l'ambiente sia identico su ogni macchina.
openapi-typescript	7.13.0	Generazione dei tipi TypeScript del client a partire dallo schema OpenAPI prodotto da FastAPI. Il frontend _G non dichiara autonomamente la forma dei dati scambiati con il backend: la deriva dal contratto pubblicato dal backend stesso. Una modifica di uno schema Pydantic si propaga così ai tipi del client e, se incompatibile con l'uso che il frontend _G ne fa, si manifesta come errore di compilazione anziché come guasto in esecuzione. Il file generato è escluso dall'analisi di ESLint, non essendo codice scritto a mano.

Nota sulle versioni degli strumenti di sviluppo. Le versioni indicate per **Docker**, **Docker Compose** e **uv** vanno lette diversamente da tutte le altre del documento. Quelle delle librerie sono fissate dai file di lock e risultano identiche su ogni macchina; queste tre no, perché nessun file del progetto le vincola: Docker e Compose sono presupposti dall'ambiente anziché dichiarati, e uv è installato dall'immagine del backend senza indicazione di versione e risolto alla versione corrente dall'integrazione_G continua. I valori riportati sono quindi quelli **in uso sull'ambiente di sviluppo del gruppo** alla data del documento, e non un vincolo_G verificabile_G: su un'altra macchina, o in un altro momento, l'installazione può risolvere versioni diverse. Per Docker il numero è quello di CLI e motore riportato da `docker --version`, che non coincide con la versione di Docker Desktop, la quale segue una numerazione propria. Fa eccezione **pnpm**, che il repository_G vincola: il valore riportato è quello dichiarato in `package.json` e non quello rilevato dall'ambiente.

Verifica automatica delle pull request_G. I controlli eseguiti dall'integrazione_G continua sono la condizione che ogni modifica deve soddisfare prima di essere integrata_G nel ramo principale. Sono organizzati in **tre processi indipendenti**, ciascuno con i propri passi:

- **frontend_G:** installazione vincolata al file di lock, ESLint, rigenerazione dei tipi dallo schema OpenAPI e confronto con il file versionato, compilazione dei tipi con **tsc**, esecuzione di Vitest con misurazione della copertura del codice_G e verifica_G delle soglie, costruzione del pacchetto di produzione con Vite;
- **backend:** installazione vincolata a **uv.lock**, Ruff nella modalità di controllo delle regole, riesportazione dello schema OpenAPI e confronto con il file versionato, esecuzione di pytest con **pytest-cov** e verifica_G della soglia;
- **contratti di dipendenza:** installazione vincolata a **uv.lock** e controllo dei contratti dichiarati per l'architettura del backend, descritti nella sezione 3.4.1.

La soglia di copertura verificata dai primi due processi è quella fissata dal Piano di Qualifica per il valore accettabile della metrica_G corrispondente; entrambi conservano il rapporto prodotto come artefatto_G del processo, così che il dato sia consultabile a posteriori e non solo nel registro di esecuzione.

L'esito negativo di un qualunque passo impedisce l'integrazione_G. Questi controlli soddisfano il requisito_G **R-4-Q-Ob**, che subordina l'integrazione_G nel ramo principale al superamento dei test_G automatici oltre che alla revisione tra pari, e rendono **verificabile** quanto la voce `openapi-typescript` afferma: perché un'incompatibilità di contratto si manifesti “come errore di compilazione” occorre che una compilazione avvenga a ogni modifica, in un ambiente che non si possa aggirare. Il controllo sul contratto è **doppio e simmetrico**: il processo del backend riesporta lo schema dal codice e pretende che coincida con `openapi.json` versionato, quello del frontend_G rigenera i tipi da quello schema e pretende che coincidano con il file versionato. Se il primo confronto fallisce, il contratto pubblicato non corrisponde più al codice che lo produce; se fallisce il secondo, lo schema è cambiato senza che il client sia stato aggiornato. In entrambi i casi la verifica_G si interrompe prima che il disallineamento raggiunga il ramo principale.

Nota sui container. Un container non è una macchina virtuale, bensì un normale processo del sistema_G operativo ospite, isolato mediante funzionalità del kernel: i *namespaces* gli attribuiscono una vista propria di processi, rete e file system, i *cgroups* ne limitano il consumo di risorse. Poiché il kernel è condiviso con la macchina ospite, anziché replicato come in una macchina virtuale, l'ingombro è sensibilmente inferiore e l'avvio richiede secondi anziché minuti.

Alternative considerate

- **Configurazione manuale dell'ambiente al posto di Docker**

Richiederebbe a ciascun membro del gruppo di installare e allineare manualmente le versioni di Node.js, di Python e delle rispettive dipendenze di sistema_G su tre famiglie di sistemi_G operativi differenti. Le configurazioni divergerebbero inevitabilmente, con la conseguenza che un comportamento osservato su una macchina non sarebbe riproducibile sulle altre — circostanza che rende la diagnosi dei guasti sproporzionatamente onerosa e vanifica il valore probatorio dei test_G.

- **Podman al posto di Docker**

Podman è compatibile con i formati e i comandi di Docker e presenta un vantaggio di sicurezza_G

non trascurabile: non richiede un processo demone privilegiato in esecuzione permanente e consente di eseguire i container senza privilegi amministrativi. È stato scartato per ragioni di diffusione, ampiezza della documentazione e familiarità pregressa dei membri del gruppo, elementi che riducono il tempo speso in attività non produttive.

- **Poly-repo al posto del mono-repo**

La separazione di frontend_G e backend in repository_G distinte è appropriata quando i due componenti sono sviluppati da gruppi diversi e rilasciati in modo indipendente. Non è il caso del progetto: le due parti sono sviluppate dallo stesso gruppo e rilasciate congiuntamente. La separazione imporrebbe due pull request_G coordinate a ogni variazione del contratto dell'API $_G$, con una finestra temporale in cui le due repository_G risultano disallineate, oltre alla duplicazione della configurazione di verifica_G .

- **Tipi TypeScript scritti a mano al posto della generazione da OpenAPI**

È l'alternativa che non aggiunge alcuno strumento, ma introduce una duplicazione del contratto dell'API $_G$ in due punti del sistema $_G$ mantenuti separatamente. Le due definizioni divergono inevitabilmente non appena il backend evolve senza che il frontend_G venga aggiornato di conseguenza; poiché i tipi TypeScript sono rimossi in compilazione, la divergenza non produce alcun errore e si manifesta come guasto silenzioso in esecuzione. La generazione a partire dallo schema elimina la duplicazione alla radice.

3 Architettura

3.1 Panoramica architetturale

La presente sezione fornisce la visione d'insieme del sistema_G: le unità che lo compongono, i confini che le separano e lo stile architetturale adottato all'interno di ciascuna. Ne fissa inoltre il **vocabolario**, che le sezioni successive impiegano senza introdurre sinonimi. Il dettaglio di ciascuna parte è rimandato a quelle sezioni: qui si stabilisce soltanto che cosa esiste, come è delimitato e secondo quale criterio è organizzato al proprio interno.

3.1.1 Visione d'insieme

Il sistema_G è composto da **due unità di deployment indipendenti**:

- un'**applicazione web a pagina singola** (*Single Page Application*), eseguita interamente nel browser dell'utente;
- un **servizio applicativo** che espone un'API_G HTTP.

Le due unità risiedono in altrettante cartelle della repository_G — **web/** e **api/** — ciascuna con il proprio gestore di pacchetti, le proprie dipendenze e la propria catena di costruzione: restano quindi due progetti distinti, costruiti ed eseguiti separatamente, e non due parti di un unico artefatto_G. Il modo in cui vengono poste in esecuzione è trattato nella sezione dedicata all'architettura di deployment.

La comunicazione fra le due unità avviene su **HTTP** secondo uno stile **REST**, ed è asimmetrica: l'applicazione web è l'unico soggetto che avvia le richieste, il servizio applicativo il solo che risponde. Le risposte delle operazioni assistite dal modello linguistico sono restituite **in streaming**: il testo raggiunge il browser a frammenti, man mano che il modello lo produce, e compare progressivamente nell'interfaccia anziché tutto insieme al termine dell'elaborazione. Il servizio non trattiene il testo per consegnarlo completo, ma lo inoltra appena disponibile; il formato con cui i frammenti viaggiano sulla connessione appartiene alla sezione sul flusso dei dati.

Il servizio applicativo è a sua volta **client di due servizi esterni**, entrambi contattati via HTTP e nessuno dei due realizzato dal gruppo:

- un **gateway LiteLLM**, che espone un'interfaccia compatibile con quella di OpenAI e per il cui tramite avviene l'inferenza del modello linguistico;
- **Tavily**, che restituisce il contenuto testuale di una pagina web il cui indirizzo è indicato dall'utente.

Il perimetro del sistema_G comprende quindi due unità realizzate dal gruppo e due dipendenze esterne raggiunte dalla sola unità server: l'applicazione web non contatta in alcun caso i due fornitori.

Non esiste persistenza lato server. Il servizio applicativo non possiede dati: non dispone di una base di dati, non conserva stato applicativo fra una richiesta e la successiva e il suo dominio_G non dichiara alcuna porta per le note o per gli utenti. I dati dell'utente restano nel browser. Il meccanismo con cui vi permangono è descritto altrove; qui rileva la sola conseguenza architetturale: il confine del sistema_G non racchiude alcun archivio, e le due unità non condividono alcuno stato persistente.

3.1.2 Stile architetturale del servizio applicativo

Il servizio applicativo adotta l'**architettura esagonale** (*Ports & Adapters*). Il criterio che la caratterizza è la direzione delle dipendenze: al centro stanno le regole del prodotto, attorno gli adattatori che le collegano al mondo esterno, e la dipendenza punta sempre verso il centro, mai in senso contrario.

Il nucleo. Il package `app/core/` contiene le regole del prodotto — i casi d'uso, i valori del dominio_G e le porte — e **non importa nulla dagli altri package**. Non conosce il framework_G web, non conosce il client HTTP, non conosce i fornitori esterni né il modulo che legge la configurazione. Un caso d'uso è una funzione che riceve i dati dell'operazione e, come ultimo parametro, la porta di cui si serve: non sa chi la implementi e non ha modo di scoprirlo.

Le porte. Le porte dichiarate sono **due**, entrambe in `app/core/ports/`. `LLMClient` dichiara la produzione incrementale di testo da parte di un modello linguistico; `ContentExtractor` dichiara l'estrazione del contenuto testuale di un indirizzo web. Sono interfacce astratte: descrivono ciò di cui il dominio_G ha bisogno, non il modo in cui un fornitore lo realizza. Accanto a ciascuna è dichiarato il tipo di errore che essa può sollevare, così che chi la consuma possa trattarne il fallimento senza conoscere l'implementazione che lo produce.

I due lati dell'esagono. Gli adattatori si dispongono su due lati opposti. In `app/api/` vive ciò che traduce una richiesta esterna in una chiamata al dominio_G: è il lato da cui l'applicazione viene invocata. In `app/infrastructure/` vive ciò che il dominio_G chiama per parlare con il mondo: è il lato che l'applicazione invoca. **Le due direzioni non si toccano direttamente:** un adattatore del primo lato non istanzia un adattatore del secondo, ma dichiara la porta di cui ha bisogno e la riceve dall'esterno. Il modulo che stabilisce quale adattatore soddisfa quale porta è uno solo — il *composition root* — ed è l'unico punto dell'applicazione in cui i due adattatori concreti sono nominati fuori dai moduli che li definiscono. Sostituire un fornitore esterno significa perciò scrivere un nuovo adattatore e cambiare una riga in quel modulo, non intervenire sul dominio_G.

Il vincolo_G non è affidato alla disciplina dei redattori. È il punto qualificante di questa architettura, e va detto esplicitamente: la direzione delle dipendenze **non è una convenzione che il gruppo si impegna a rispettare**, ma un vincolo_G dichiarato e verificato_G automaticamente. I contratti sono scritti in `api/.importlinter` e controllati staticamente dallo strumento `import-linter`, che ispeziona il grafo delle importazioni senza eseguire il codice. Sono quattro e stabiliscono che il nucleo non importi il framework_G web, il client HTTP, il client del servizio di estrazione, la libreria di configurazione né i package degli adattatori; che i livelli siano ordinati dall'esterno verso il centro; che gli adattatori non si importino fra loro; che il nucleo non importi il modulo di configurazione. Il controllo è un passo dell'integrazione_G continua, non un'abitudine locale: un'importazione che invertisse la dipendenza non sopravviverebbe fino alla revisione tra pari, perché farebbe fallire la verifica_G automatica prima.

Il beneficio ottenuto. Ciò che l'esagono restituisce, e che è verificabile_G nel codice, è la **provabilità del dominio_G in isolamento**: un caso d'uso si esercita senza montare HTTP e senza rete, passandogli al posto della porta un doppio che ne rispetta il contratto. I test_G dei casi d'uso del prodotto sono scritti esattamente così, e non istanziano alcun adattatore. La proprietà non è un corollario teorico dello stile adottato: discende dal fatto che il tipo del parametro sia

la porta e mai l'adattatore, condizione che i contratti di dipendenza rendono impossibile violare inavvertitamente.

Lo stile qui dichiarato — un servizio applicativo unico, organizzato a esagono — è stato adottato in alternativa alla scomposizione in microservizi e al monolite a strati tradizionale. L'argomentazione del confronto, con i costi e i benefici di ciascuna alternativa, è riportata nelle sezioni 3.2.2 e 3.2.3.

3.1.3 Stile architetturale dell'applicazione web

L'applicazione web è organizzata **a componenti, con stato condiviso esterno all'albero e flusso dei dati unidirezionale**.

I componenti React compongono l'interfaccia per annidamento: ciascuno riceve dal genitore i dati che deve presentare e ne restituisce la rappresentazione. Lo stato realmente condiviso non risiede però nell'albero dei componenti: vive fuori di esso, in **due store distinti**, che i componenti interrogano direttamente senza doverlo trasmettere lungo la gerarchia. Il flusso resta unidirezionale: un'interazione dell'utente invoca un'azione dello store, l'azione aggiorna lo stato, i componenti sottoscritti alla porzione modificata si ridisegnano. Non esiste un percorso inverso in cui un componente modifichi la vista di un altro.

La regola seguita nel collocare lo stato è graduale: **stato locale al componente per difetto**, sollevato al genitore comune quando due componenti vicini devono concordare, **promosso a store soltanto quando componenti lontani nell'albero devono condividerlo**. Lo store non è quindi il deposito di tutto lo stato dell'applicazione: le condizioni puramente locali di un componente — l'apertura di un pannello, il testo in corso di digitazione in un campo, la voce dell'elenco in fase di rinomina — restano nel componente che le possiede, e non se ne ha traccia altrove.

I componenti si sottoscrivono a **single porzioni** di stato, non allo stato intero: ciascuno store espone un selettore per ogni porzione osservabile, e il componente che ne dichiara uno viene ridisegnato solo quando quella porzione cambia. È la proprietà che rende sostenibile lo streaming: mentre il testo arriva a frammenti e la porzione che lo raccoglie cambia molte volte al secondo, si ridisegna la sola parte di interfaccia che presenta l'esito in arrivo, mentre l'editor — sottoscritto a una porzione distinta, il testo della nota — non viene toccato. Una sottoscrizione allo stato nel suo complesso, o a un oggetto che ne aggrega più porzioni, riporterebbe il ridisegno sull'intera area di lavoro a ogni frammento ricevuto.

I due store si dividono le responsabilità in questo modo: `useEditorStore` custodisce lo stato della sessione di scrittura, cioè il testo corrente, la selezione, l'esito in arrivo dal servizio applicativo, la condizione di errore e di avanzamento e la modalità di visualizzazione; `useNotesStore` custodisce la raccolta delle note e l'indicazione di quale sia quella corrente. La loro composizione interna appartiene alla sezione sull'architettura logica dell'applicazione web.

3.1.4 Vocabolario

I termini fissati nella tabella 6 sono impiegati con questo significato in tutto il capitolo. Le sezioni successive vi si attengono e non ne adottano sinonimi.

Tabella 6: Vocabolario architetturale

Termine	Definizione
Porta	Interfaccia astratta dichiarata dal nucleo, che descrive ciò di cui il dominio _G ha bisogno dall'esterno senza indicare come venga realizzato.

Termine	Definizione
Adattatore primario	Modulo che traduce una richiesta proveniente dall'esterno in una chiamata al dominio _G ; risiede in <code>app/api/</code> .
Adattatore secondario	Modulo che realizza una porta traducendo la chiamata del dominio _G nel protocollo di un servizio esterno; risiede in <code>app/infrastructure/</code> .
Nucleo (dominio_G)	Insieme delle regole del prodotto, contenuto in <code>app/core/</code> , che non importa nulla dagli altri package del servizio applicativo.
Caso d'uso	Funzione del nucleo che realizza una singola operazione del prodotto, ricevendo i dati dell'operazione e, come ultimo parametro, la porta di cui si serve.
Composition root	Unico modulo che stabilisce quale adattatore soddisfa quale porta, e unico punto in cui gli adattatori concreti sono nominati fuori dai moduli che li definiscono.
Store	Contenitore di stato condiviso dell'applicazione web, esterno all'albero dei componenti, che espone lo stato in lettura e le azioni che lo modificano.
Selettore	Funzione che estrae da uno store una singola porzione di stato e alla quale un componente si sottoscrive; determina quando quel componente viene ridisegnato.

3.2 Architettura di deployment

3.2.1 Unità di esecuzione e configurazione

3.2.2 Confronto con l'architettura a microservizi

3.2.3 Confronto con il monolite a strati

3.3 Architettura del frontend

L'applicazione web è organizzata secondo il principio dichiarato in §3.1.3: componenti React che compongono l'interfaccia per annidamento, stato condiviso esterno all'albero in due store distinti, flusso dei dati unidirezionale. La presente sezione descrive l'articolazione interna in sei aspetti: organizzazione dei sorgenti, scomposizione in componenti, gestione dello stato, integrazione dell'editor, accesso al servizio applicativo e persistenza lato client.

Due precisazioni preliminari. Primo: il «flusso unidirezionale» vale a livello generale — React legge dallo store, le azioni lo modificano, i componenti sottoscritti si ridisegnano. Secondo: l'integrazione con CodeMirror, descritta in §3.3.4, è un'eccezione controllata. L'API di CodeMirror è imperativa, richiede una sincronizzazione bidirezionale tra l'istanza dell'editor e lo store. La sezione spiega come questa eccezione è gestita in modo da non rompere il principio generale.

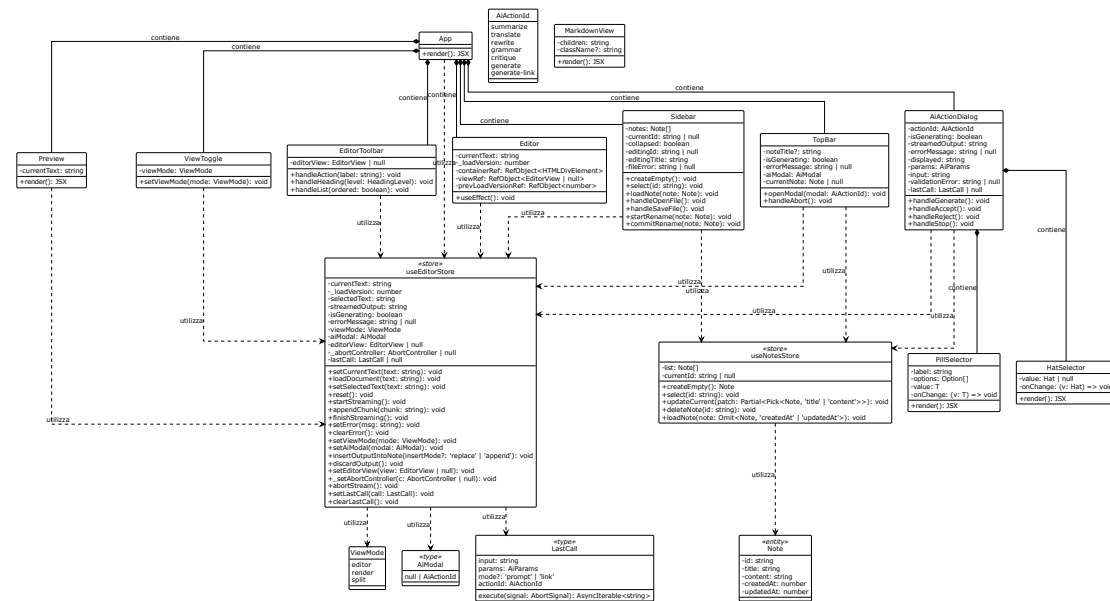


Figura 1: Diagramma UML 3.3-1 - Componenti UI e Store

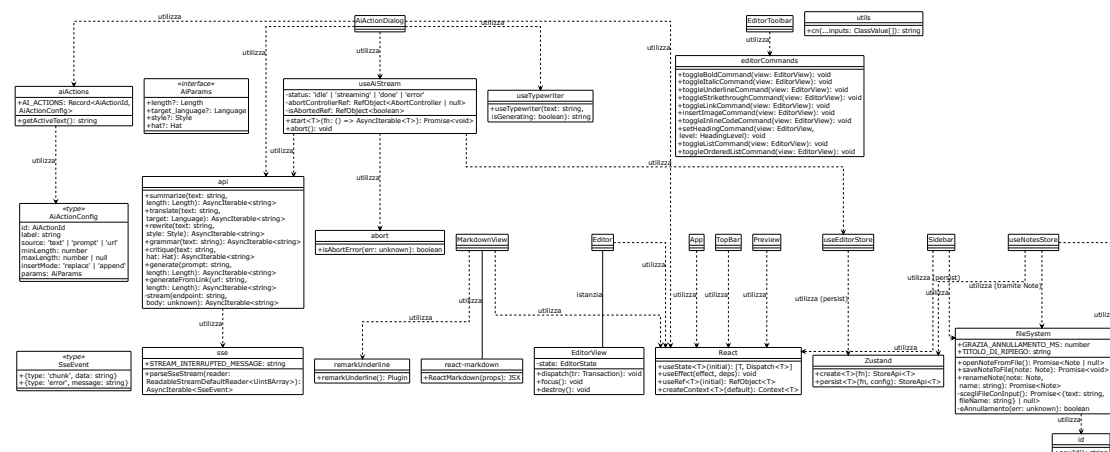


Figura 2: Diagramma UML 3.3-2 -Hooks, Librerie e Dipendenze Esterne

3.3.1 Organizzazione dei sorgenti

La cartella `web/src/` è organizzata per **ruolo**, non per feature. Ogni cartella contiene file che condividono la stessa responsabilità architetturale, indipendentemente dalla funzione di prodotto. Nel livello radice di `src/` risiedono i file di ingresso dell'applicazione: `main.tsx` (entry point del bundle, che monta l'applicazione nel DOM), `App.tsx` (componente radice, che definisce il layout principale e la griglia dinamica dell'area di lavoro), e `index.css` (foglio di stile globale, che include le direttive di Tailwind CSS e le variabili di tema). I moduli organizzati per ruolo sono invece distribuiti nelle seguenti cartelle:

- **components/**: componenti React. **ui/** contiene primitivi (**alert**, **button**, **textarea**) realizzati con **radix-ui** e **class-variance-authority**; i file direttamente in **components/** contengono i componenti compositi: **Sidebar**, **TopBar**, **Editor**, **EditorToolbar**, **Preview**, **ViewToggle**, **AiActionDialog**, **MarkdownView**. All'interno di **AiActionDialog.tsx** sono definiti i componenti interni **PillSelector** e **HatSelector**, non esportati separatamente.
- **hooks/**: hook personalizzati: **useAiStream** (logica di streaming), **useTypewriter** (effetto di battitura progressiva). L'hook **useAiStream** soddisfa R-109-F-De (indicatore di avanzamento per operazioni lunghe) e R-79-F-De (annullamento di una richiesta in corso).
- **store/**: i due store Zustand: **useEditorStore.ts** (stato della sessione di scrittura), **notes.ts** (raccolta delle note).
- **lib/**: utility e servizi: **api.ts** (facade API), **sse.ts** (parser SSE), **fileSystem.ts** (importazione/esportazione), **abort.ts** (gestione abort), **id.ts** (generazione ID), **editorCommands.ts** (comandi CodeMirror), **aiActions.ts** (registry delle azioni AI), **remarkUnderline.ts** (plugin Remark), **utils.ts** (funzione cn).
- **types/**: definizioni dei tipi. **api.ts** è auto-generato da **openapi-typescript** dallo schema OpenAPI; **models.ts** lo re-esporta isolando il resto dell'applicazione dalla struttura generata.
- **test/**: contiene il **setup.ts** dei test (Vitest, Testing Library, jsdom), soddisfacendo R-2-Q-Ob e R-3-Q-Ob (test automatici e copertura).
- **assets/**: risorse statiche: **hero.png**, **react.svg**, **vite.svg**.

Questa organizzazione rende prevedibile la collocazione di un nuovo file. Facilita inoltre la verifica automatica delle dipendenze: un linter può controllare che i componenti non importino direttamente i tipi generati dall'OpenAPI senza passare dal barrel, e che gli store non importino i componenti.

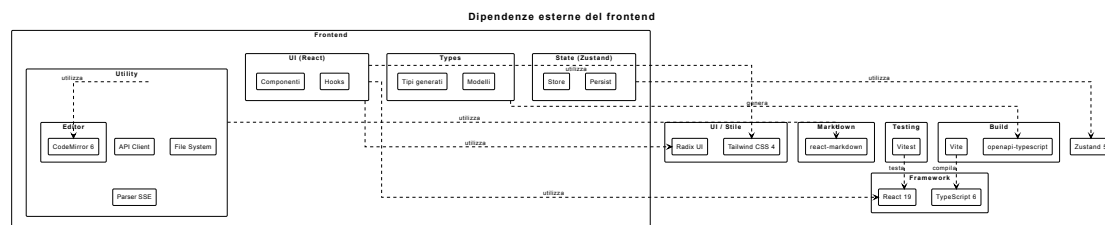


Figura 3: Diagramma UML 3.3.1 - Organizzazione dei sorgenti del frontend

3.3.2 Scomposizione in componenti

L'albero dei componenti ha radice in **App**, che definisce il layout a tre colonne e la griglia dinamica dell'area di lavoro.

App legge **viewMode** da **useEditorStore** tramite il selettore **useViewMode**. La struttura del **main** cambia in base al valore: in modalità **split** viene resa una griglia a due colonne (**md:grid-cols-2**); in modalità **editor** o **render** un singolo pannello a tutta larghezza. La presenza di **Editor** e **Preview** è determinata dalla stessa condizione. Questa logica soddisfa i requisiti R-50-F-Ob, R-51-F-Ob e R-52-F-Ob (visualizzazioni esclusiva editor, esclusiva render e combinata).

I componenti compositi sono i seguenti:

- **Sidebar:** elenca le note da `notes` tramite `useNotesList`. Espone un pulsante per creare una nuova nota (`createEmpty`) che soddisfa R-82-F-Ob e R-114-F-Ob. Gestisce la selezione (`select`). Ogni voce mostra il titolo e supporta la ridenominazione tramite doppio click, che attiva un campo di input (R-94-F-De). Offre i pulsanti «Apri file...» e «Salva file», che invocano `openNoteFromFile` e `saveNoteToFile` (§3.3.6), soddisfacendo R-85-F-Ob e R-88-F-Ob. Un pulsante di collasso riduce la barra a una colonna stretta.

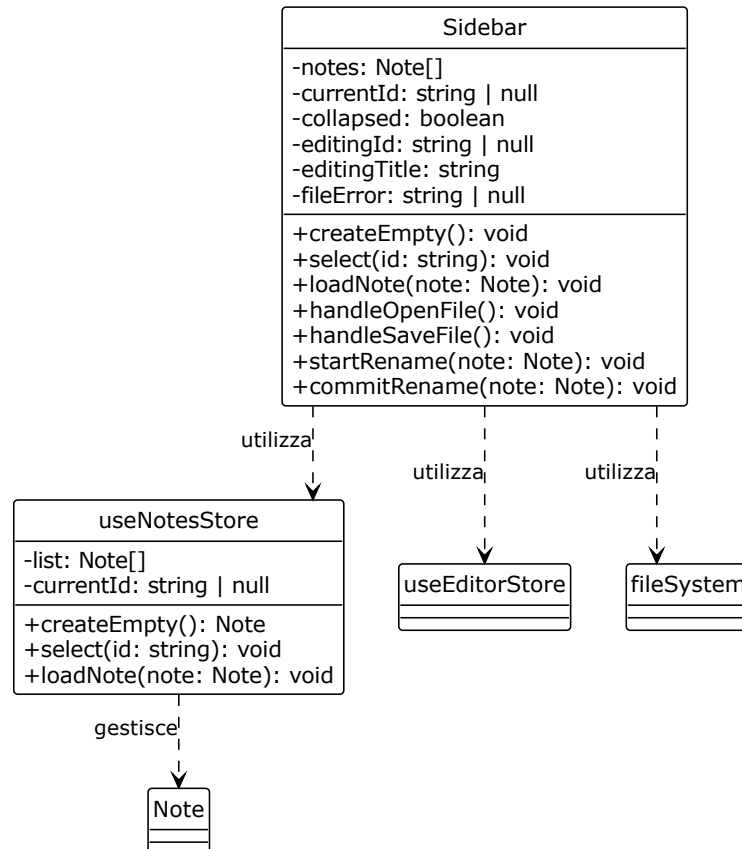


Figura 4: Diagramma UML 3.3.2-1 - Sidebar

- **TopBar:** presenta il titolo della nota corrente (da `useCurrentNote`) e i pulsanti delle azioni AI. Alla pressione di un pulsante, invoca `setAiModal` per aprire il dialogo corrispondente. Durante lo streaming (`isGenerating`), mostra un pulsante di interruzione (`abortStream`) e un indicatore di caricamento (R-109-F-De). In caso di errore, visualizza `errorMessage` in un alert (R-110-F-Ob).

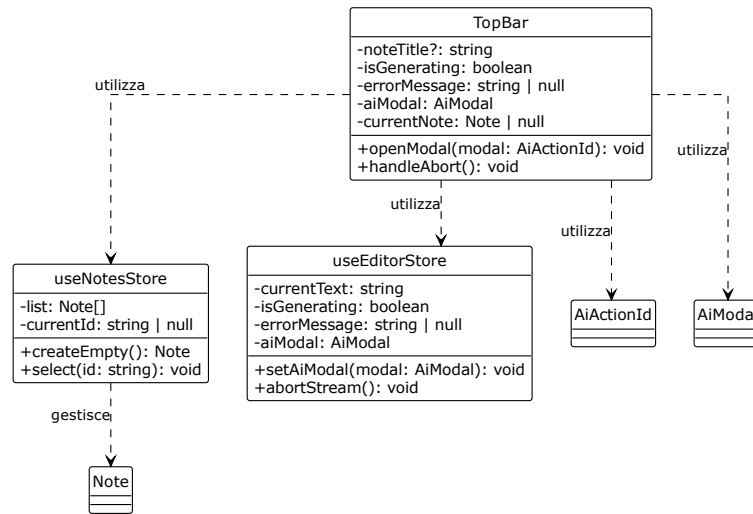


Figura 5: Diagramma UML 3.3.2-2 - TopBar

- **ViewToggle**: tre pulsanti che impostano `viewMode` su `editor`, `split` o `render`. È sempre visibile, in qualsiasi modalità, per consentire il passaggio diretto a una vista diversa.

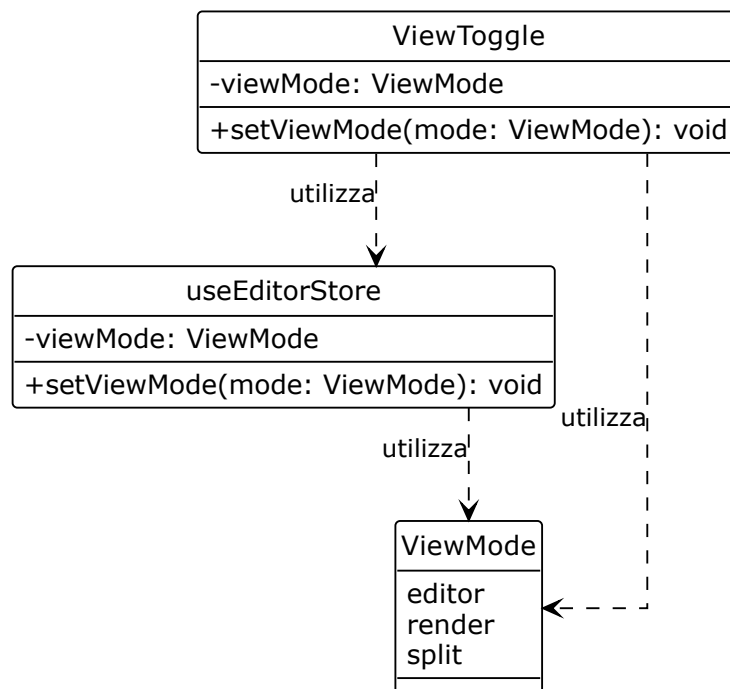


Figura 6: Diagramma UML 3.3.2-3 - ViewToggle

- **EditorToolbar**: contiene i controlli per la formattazione del testo Markdown. Le operazioni corrispondono ai casi d'uso UC8-UC10 (grassetto, R-9-F-Ob, R-10-F-Ob), UC11-UC13 (corsivo, R-11-F-Ob, R-12-F-Ob), UC14-UC16 (sottolineato, R-13-F-Ob, R-14-F-Ob), UC17-UC19 (barrato, R-15-F-De, R-16-F-De), UC20 (inserimento titolo, R-17-F-Ob), UC22 (rimozione titolo, R-18-F-Ob), UC23 (inserimento sottotitolo, R-19-F-Ob), UC25 (rimozione sottotitolo, R-20-F-Ob), UC26 (inserimento elenco, R-21-F-De, R-22-F-De), UC28 (rimozione elenco, R-23-F-De), UC38 (inserimento link, R-30-F-De), UC49 (inserimento immagine, R-39-F-De), e UC52/UC53 (attivazione/disattivazione auto-completamento, R-42-F-De, R-43-F-De). Ogni comando invoca la corrispondente funzione da `lib/editorCommands.ts` sull'istanza `editorView` conservata nello store.

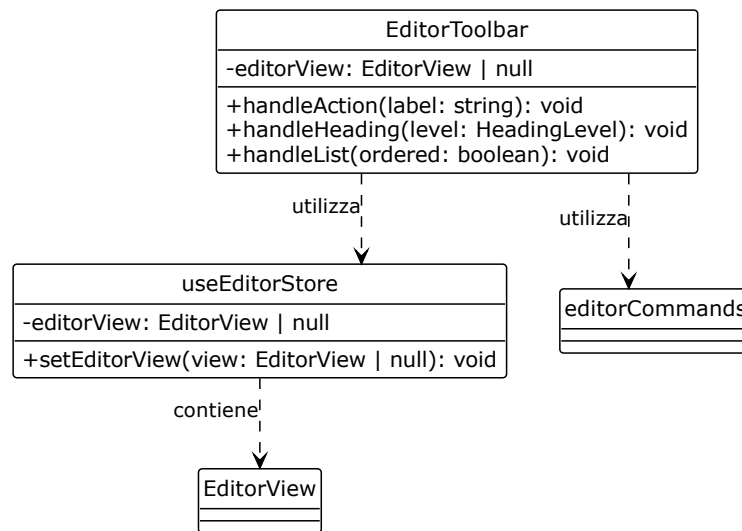


Figura 7: Diagramma UML 3.3.2-4 - EditorToolbar

- **Editor**: il componente che istanzia CodeMirror e gestisce la sincronizzazione bidirezionale con **useEditorStore** (§3.3.4). Soddisfa i requisiti R-1-F-Ob (inserimento testo) e R-2-F-Ob (visualizzazione grafica del Markdown).

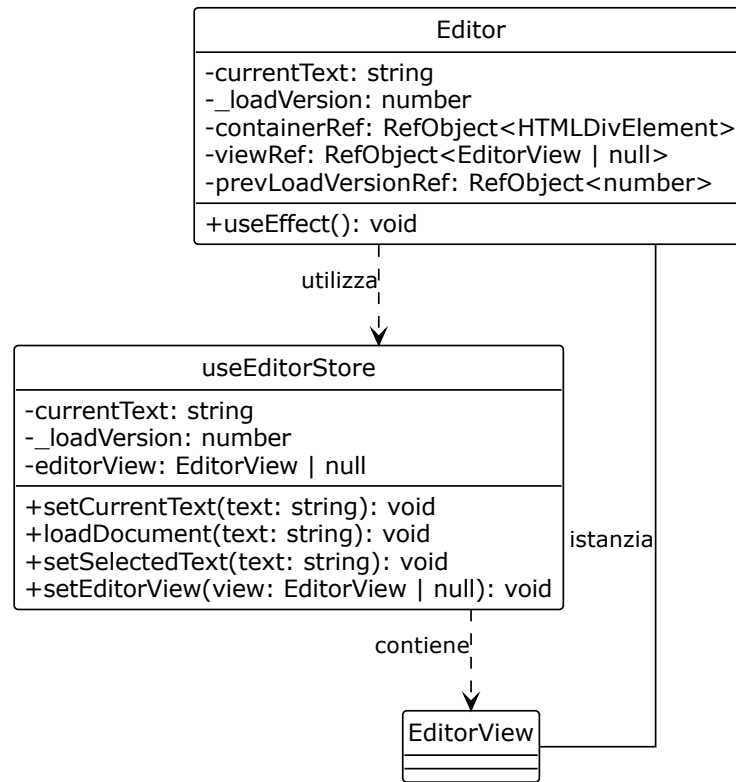


Figura 8: Diagramma UML 3.3.2-5 - Editor

- **Preview:** rende il contenuto Markdown della nota corrente usando **MarkdownView**. Applica i plugin `remark-gfm`, `remark-math`, `rehype-katex` e `rehype-highlight`. Soddisfa R-2-F-Ob e R-113-F-Ob.

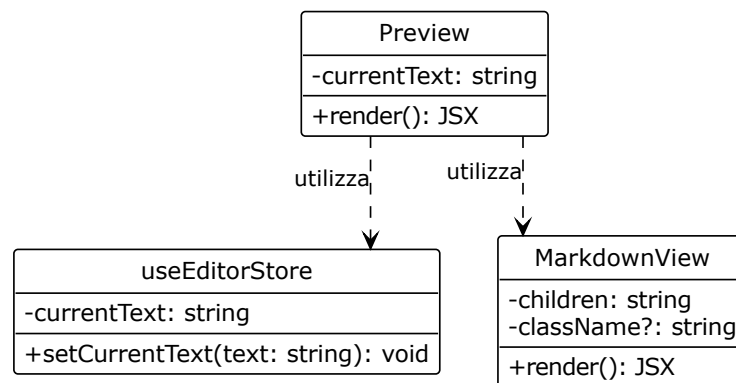


Figura 9: Diagramma UML 3.3.2-6 - Preview

- **MarkdownView**: componente condiviso da **Preview** e dall'anteprima dell'**AiActionDialog**. Applica la stessa pipeline Markdown a entrambi i contesti, garantendo coerenza visiva. Non abilita HTML raw. Utilizza **remarkUnderline** per supportare la sintassi `++sottolineato++`.

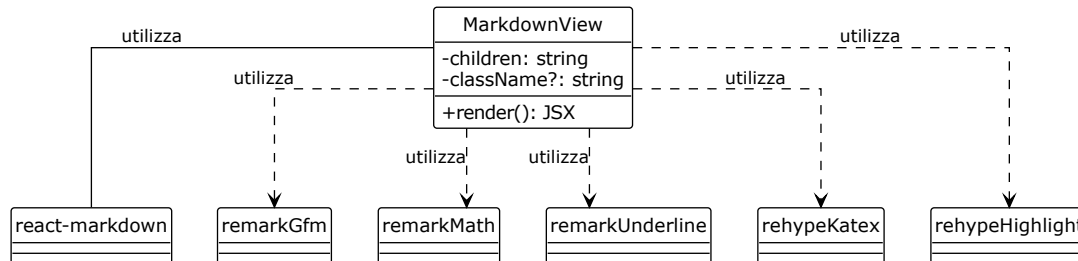


Figura 10: Diagramma UML 3.3.2-7 - MarkdownView

- **AiActionDialog**: il dialogo modale che si apre quando `aiModal` non è null. La sua implementazione è articolata:
 - **Registry delle azioni**: le azioni AI sono registrate in `lib/aiActions.ts` (pattern *Registry*, §3.5). Il registry mappa ciascun `AiActionId` alla funzione API, al titolo del dialogo, ai campi del form, alla sorgente del testo (`text`, `prompt` o `url`), alla lunghezza minima e massima del testo in input, e alla modalità di inserimento dell'output (`replace` per trasformazioni, `append` per generazioni). Il dialogo soddisfa R-54-F-Ob (accesso al LLM).
 - **Funzione `getActiveText`**: determina il testo da sottoporre all'AI. Se l'utente ha una selezione attiva in `selectedText`, viene usata quella. Altrimenti, l'intero `currentText`. La logica è centralizzata per garantire uniformità tra le azioni.
 - **Parametri e validazione**: il dialogo presenta un campo di input (textarea per prompt, input per URL, o nessun campo per le azioni sul testo) e selettori per i parametri specifici di ciascuna azione. I selettori soddisfano R-56-F-De (tipologia di riassunto), R-58-F-Ob (lingua di destinazione), R-60-F-Ob (stile di riscrittura), R-64-F-Ob (prospettiva di analisi), R-73-F-De (lunghezza del testo generato) e R-74-F-De (URL come fonte). I selettori sono `PillSelector` e `HatSelector`, definiti all'interno del file `AiActionDialog.tsx`. La validazione controlla che il testo in input soddisfi `minLength` e `maxLength` (R-81-F-Ob), e che i parametri obbligatori siano selezionati. Gli errori di validazione sono visualizzati in un alert (R-110-F-Ob).
 - **Stato locale**: il dialogo mantiene uno stato locale (`input`, `params`, `validationError`, `lastCall`) gestito tramite `useState`. `lastCall` conserva l'input e i parametri dell'ultima chiamata. Il pulsante di generazione mostra «Rigenera» invece di «Genera» quando i parametri non sono cambiati (R-78-F-De).
 - **Anteprima e accettazione**: l'output in streaming è visualizzato in un'anteprima che usa `MarkdownView` e l'effetto di battitura `useTypewriter`. Il pulsante «Accetta» invoca `insertOutputIntoNote` (R-76-F-Ob). «Rifiuta» invoca `discardOutput` (R-77-F-De). Il pulsante «Accetta» è disabilitato finché l'output è vuoto, in generazione, o nel caso particolare di `grammar` che restituisce `NO_ERRORS_MARKER` (nessun errore rilevato, R-62-F-De).

- **Terminazione:** la chiusura del dialogo tramite overlay o tasto ESC invoca `handleReject`, che abortisce lo streaming e scarta l'output.
- **Esaustività del contratto:** il file `AiActionDialog.tsx` contiene vincoli di tipo che garantiscono che tutti i valori enumerati del contratto OpenAPI siano presenti nei selettori. La riga `type LanguagesCoverContract = Exclude<Language, (typeof LANGUAGES)[number]['value']>` produce un errore di compilazione se il backend aggiunge una lingua che il frontend non espone, o se il frontend espone una lingua che il backend non prevede. La divergenza tra contratto e interfaccia diventa una mancata compilazione, non un difetto in produzione.

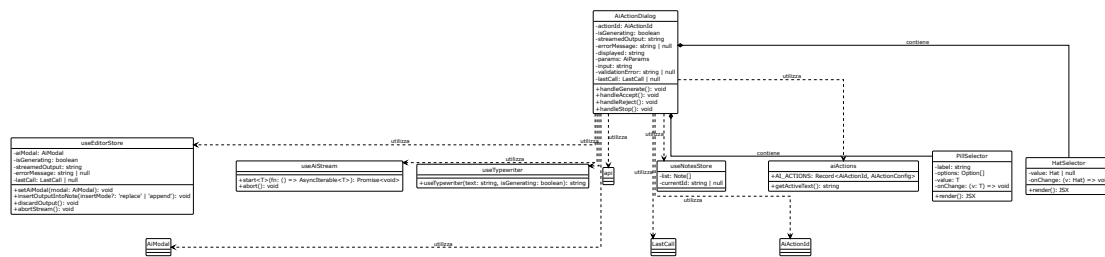


Figura 11: Diagramma UML 3.3.2-8 - AiActionDialog

3.3.3 Gestione dello stato

Lo stato condiviso è custodito in due store Zustand. Entrambi rispettano la regola dei **selettori atomici** (descritta in §3.5): ogni store espone selettori che estraggono una singola porzione di stato; i componenti si sottoscrivono solo a questi, mai allo stato aggregato. Un componente viene ridisegnato solo quando la porzione a cui è sottoscritto cambia. Questa proprietà rende sostenibile lo streaming: `Preview`, sottoscritto a `useCurrentText`, non viene ridisegnato a ogni chunk ricevuto; solo il componente che presenta `streamedOutput` viene aggiornato.

La regola di collocazione dello stato è graduale: stato locale al componente per difetto, sollevato al genitore quando due componenti vicini devono concordare, promosso a store soltanto quando componenti lontani nell'albero devono dividerlo. Lo store non è il deposito di tutto lo stato: le condizioni puramente locali di un componente restano nel componente che le possiede.

useEditorStore. Custodisce lo stato della sessione di scrittura e delle interazioni AI:

- **currentText**: il testo corrente della nota, aggiornato da **setCurrentText** a ogni modifica dell'editor.
- **_loadVersion**: contatore incrementato a ogni caricamento di una nuova nota. Serve a invalidare le operazioni asincrone in corso.
- **selectedText**: il testo selezionato dall'utente nell'editor (R-3-F-Ob), aggiornato da **setSelectedText** tramite un listener di CodeMirror.
- **streamedOutput**: il testo in arrivo dal servizio applicativo, accumulato chunk per chunk da **appendChunk**.
- **isGenerating**: flag che indica se uno streaming è in corso. **startStreaming** lo imposta a **true**; **finishStreaming** e **setError** lo imposta a **false**.

- **errorMessage**: messaggio di errore da visualizzare all'utente (R-110-F-Ob); **setError** lo imposta e disabilita **isGenerating**.
- **viewModel**: la modalità di visualizzazione dell'area di lavoro (**editor**, **render**, **split**), impostata da **setViewModel**.
- **aiModal**: l'identificatore dell'azione AI attualmente aperta nel dialogo, impostato da **setAiModal**.
- **editorView**: riferimento all'istanza di **CodeMirror**, utilizzato per operazioni di manipolazione diretta. **setEditorView** lo imposta al montaggio dell'editor.
- **_abortController**: riferimento all'**AbortController** dello streaming in corso.
- **lastCall**: conserva l'ultima richiesta AI effettuata, con input, parametri, modalità, identificatore dell'azione e la funzione eseguibile. È utilizzato per la rigenerazione (R-78-F-De) e per la gestione dello stato «Rigenera» nel dialogo. **setLastCall** e **clearLastCall** ne gestiscono il ciclo di vita.

L'azione **abortStream** interrompe la comunicazione tramite **abort()** sul controller (R-79-F-De) e, contemporaneamente, spegne l'indicatore di attesa (**isGenerating** a **false**). La proprietà è intenzionale: R-109-F-De richiede che l'indicatore di avanzamento copra l'intera durata percepita dell'operazione; al momento dell'annullamento, la durata percepita termina immediatamente, e l'indicatore si spegne senza attendere che il loop di streaming si accorga dell'interruzione.

L'azione **insertOutputIntoNote** determina la semantica di inserimento in base al parametro **insertMode**:

- **replace**: sostituisce il **selectedText** se presente e se ancora trovabile in **currentText**; altrimenti sostituisce l'intero **currentText**. È la semantica usata per le azioni di trasformazione (riassunto, riscrittura, traduzione, grammatica, analisi).
- **append**: accoda l'output alla fine di **currentText**, aggiungendo un separatore **\n\n** se necessario. È la semantica usata per la generazione da prompt o da link.

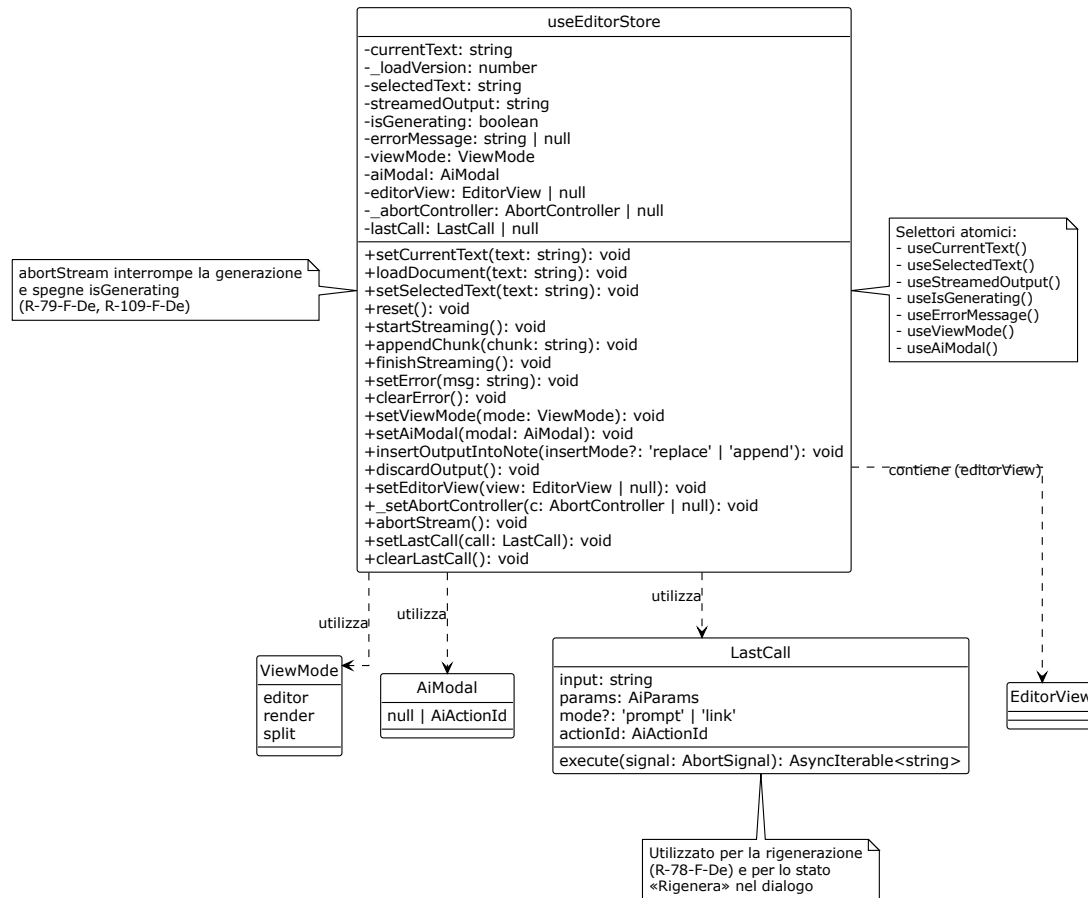


Figura 12: Diagramma UML 3.3.3-1 - useEditorStore

notes. Custodisce la raccolta delle note e l'indicazione di quale sia quella corrente:

- **list:** array di oggetti Note (con id, title, content, createdAt, updatedAt).
- **currentId:** id della nota correntemente selezionata, o null.
- **createEmpty:** crea una nuova nota con titolo «Senza titolo» e contenuto vuoto (R-82-F-Ob, R-114-F-Ob). Se una nota è già aperta, salva il suo contenuto corrente in useEditorStore prima di crearne una nuova. Incrementa _loadVersion e imposta currentText a ". Soddisfa UC74.1.
- **select:** cambia la nota corrente. Prima di cambiare, salva il contenuto corrente della nota attuale in list; poi carica il contenuto della nuova nota in useEditorStore tramite loadDocument.
- **updateCurrent:** aggiorna title e/o content della nota corrente, aggiornando updatedAt.
- **deleteNote:** rimuove la nota dall'elenco (R-91-F-De). Se la nota eliminata era quella corrente, seleziona la prima nota rimanente, se presente.
- **loadNote:** carica una nota proveniente da un'importazione (R-85-F-Ob), aggiornandola se già presente (per id) o aggiungendola se nuova.

La sincronizzazione tra i due store è gestita dalle azioni di **notes**: ogni volta che l'utente cambia nota o ne crea una nuova, il contenuto della nota corrente viene prima salvato in **useEditorStore** e poi la nuova nota viene caricata. Il flag **_loadVersion** impedisce che aggiornamenti asincroni dell'editor sovrascrivano il contenuto di una nota diversa da quella che l'utente stava effettivamente modificando.

Persistenza dello store. **notes** utilizza il middleware **persist** di Zustand (pattern descritto in §3.5) per salvare automaticamente **list** e **currentId** in **localStorage**. Il middleware serializza lo stato su ogni modifica e lo ripristina all'avvio dell'applicazione. Durante il ripristino (**onRehydrateStorage**), **loadDocument** viene chiamato per caricare il contenuto della nota corrente nell'editor. Questa persistenza soddisfa R-80-F-Ob, R-84-F-Ob, R-87-F-Ob, R-90-F-Ob, R-93-F-De, R-96-F-De e UC75.

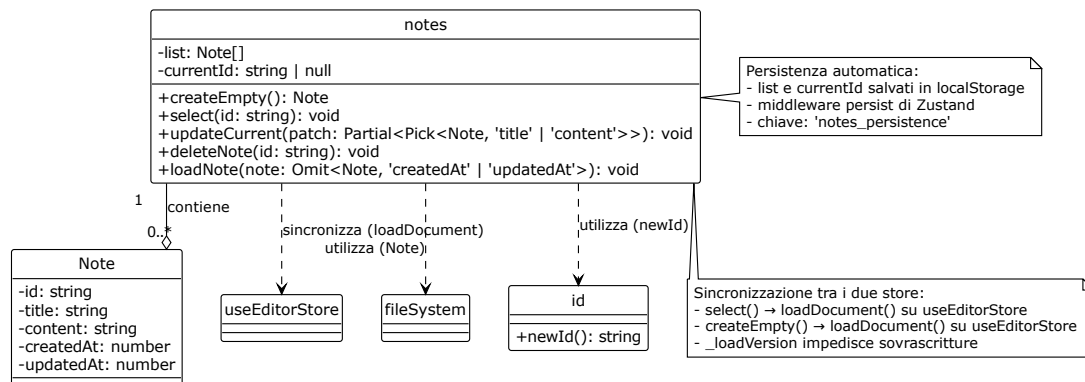


Figura 13: Diagramma UML 3.3.3-2 - notes

3.3.4 Integrazione dell'editor

L'editor è realizzato con CodeMirror 6. È integrato con lo store tramite un meccanismo di sincronizzazione bidirezionale controllata. Come anticipato in apertura, questa è un'eccezione al principio del flusso unidirezionale, dovuta all'API imperativa di CodeMirror.

Creazione dell'istanza. L'istanza di **EditorView** viene creata una sola volta, al montaggio del componente **Editor**. Le estensioni sono configurate in un array stabile per evitare il ricaricamento dell'editor:

- **lineNumbers()**: numeri di riga.
- **history()**: cronologia di undo/redo (R-7-F-De, R-8-F-De).
- **markdown()**: supporto alla sintassi Markdown (R-112-F-Ob).
- **closeBrackets()**: chiusura automatica di parentesi e virgolette (R-42-F-De, R-43-F-De).
- **EditorView.lineWrapping**: a capo automatico delle righe lunghe.
- **search({ top: true })**: barra di ricerca.
- **keymap.of([...defaultKeymap, ...historyKeymap, ...closeBracketsKeymap, ...searchKeymap, { key: 'Mod-k', run: toggleLinkCommand }])**: scorciatoie da tastiera.

L'istanza viene salvata nello store tramite `setEditorView` e rimossa al momento dello smontaggio.

Sincronizzazione store → editor. Quando il contenuto della nota cambia nello store (`currentText`), l'editor deve rifletterlo. Il meccanismo:

1. Il componente `Editor` si sottoscrive a `useCurrentText` e a `_loadVersion`.
2. Quando il valore cambia, il componente verifica se il nuovo testo è diverso da quello attualmente visualizzato (`view.state.doc.toString()`).
3. In caso affermativo, costruisce una `Transaction` che sostituisce l'intero documento e la invia all'editor tramite `view.dispatch()`.
4. La transazione è annotata con `StoreSync.of(true)`. Questa annotazione segnala all'`updateListener` che la modifica proviene dallo store e non deve essere re-inviata allo store, evitando il ciclo.
5. Se il cambiamento è dovuto a `_loadVersion` (caricamento di una nuova nota), la transazione riceve anche `Transaction.addToHistory.of(false)`. Questa annotazione impedisce che il caricamento di una nota venga registrato nella cronologia di undo, soddisfacendo R-7-F-De e R-8-F-De.

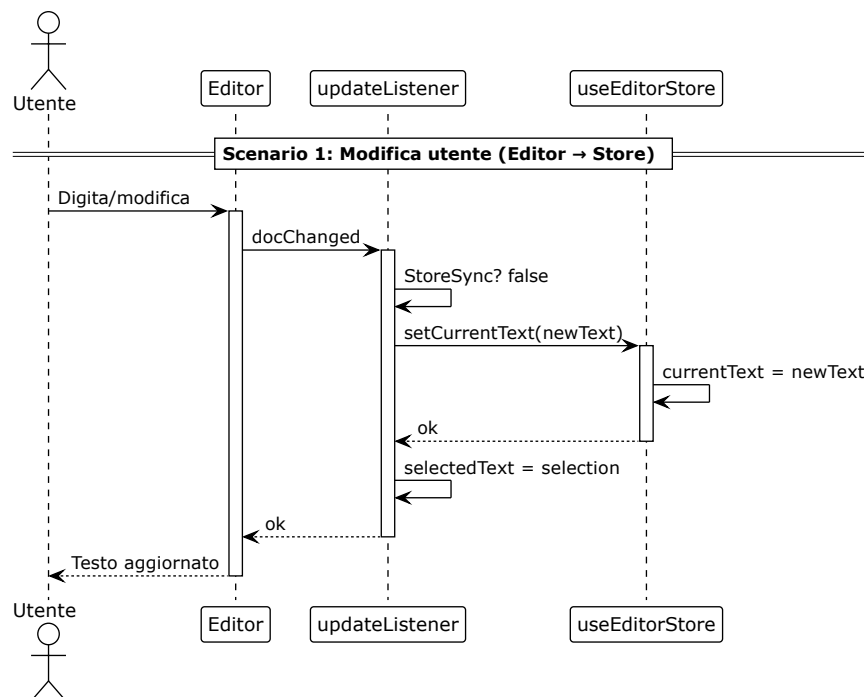


Figura 14: Diagramma UML 3.3.4-1 - Sincronizzazione store → editor

Sincronizzazione editor → store. Quando l'utente modifica il testo nell'editor, lo store deve aggiornare `currentText`. Il meccanismo:

1. L'estensione `EditorView.updateListener` intercetta ogni transazione che modifica il documento.
2. Se la transazione ha l'annotazione `StoreSync`, viene ignorata.
3. Se la transazione ha modificato il documento (`update.docChanged`), l'estensione estrae il nuovo testo completo (`update.state.doc.toString()`) e invoca `setCurrentText` con esso.
4. Lo stesso listener monitora anche i cambiamenti di selezione (`update.selectionSet`) e aggiorna `selectedText` nello store (R-3-F-Ob).

Evitare il ciclo. La sincronizzazione bidirezionale potrebbe generare un ciclo: modifica editor → `setCurrentText` → re-render del componente → `view.dispatch()` → modifica editor → ... Il ciclo è evitato da due meccanismi:

1. **Confronto:** il dispatch store→editor è eseguito solo se il testo corrente dell'editor è diverso da `currentText`. L'`updateListener` editor→store esegue `setCurrentText` solo se il testo appena modificato è diverso dal valore già in store.
2. **Annotazione `StoreSync`:** le transazioni store→editor sono marcate; l'`updateListener` le ignora, interrompendo il ciclo alla radice.

Selezione. La selezione dell'utente nell'editor è monitorata dall'estensione di `update`, che estrae l'intervallo selezionato e invoca `setSelectedText`. Il valore è usato da `getActiveText` per determinare il testo da sottoporre all'AI (R-3-F-Ob).

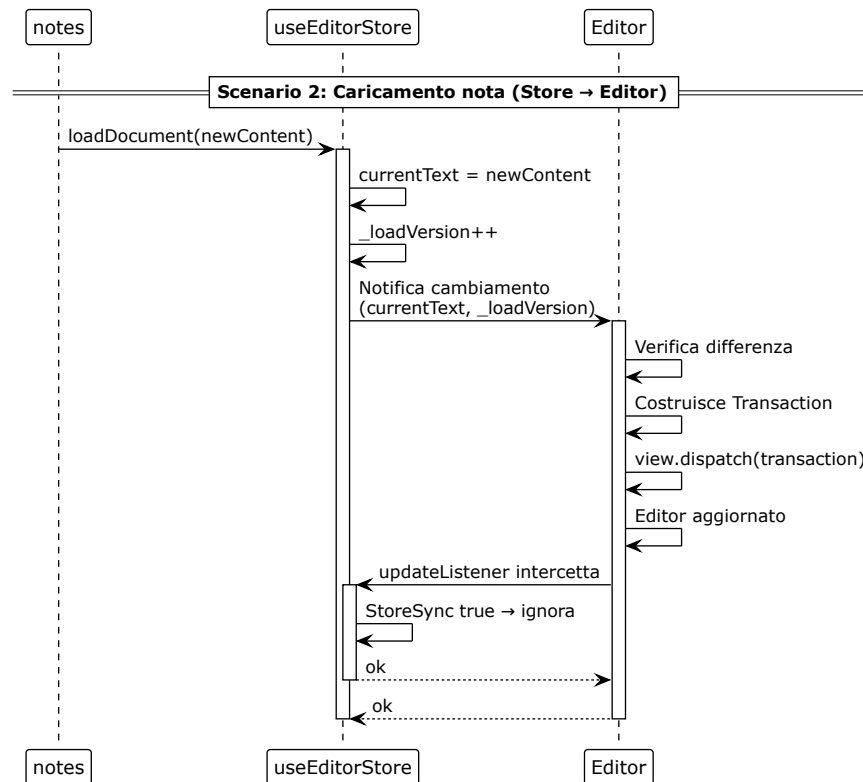


Figura 15: Diagramma UML 3.3.4-2 - Sincronizzazione editor → store

3.3.5 Accesso al servizio applicativo

L'accesso al servizio applicativo è incapsulato in tre livelli: una **facade API** in `lib/api.ts` (pattern *Facade*, §3.5), un **parser SSE** in `lib/sse.ts`, e un **hook custom** `useAiStream` in `hooks/useAiStream.ts`.

Facade API. La facade espone una funzione per ciascuna operazione del servizio applicativo: `summarize`, `translate`, `rewrite`, `grammar`, `critique`, `generate`, `generateFromLink`. Ogni funzione costruisce il corpo della richiesta con i tipi definiti in `types/models.ts` e invoca la funzione interna `stream`, che:

1. Esegue una richiesta POST all'endpoint corrispondente (con base URL da variabile d'ambiente `VITE_API_BASE_URL`).
2. Se lo stato HTTP non è 2xx, estrae `detail` dal corpo JSON e solleva un errore.
3. Se la risposta ha un corpo leggibile, crea un reader e itera sugli eventi restituiti da `parseSseStream`.
4. La funzione ritorna un `AsyncIterable<string>` che produce i frammenti in sequenza.

L'uso dei tipi generati da OpenAPI (`types/api.ts`) garantisce la conformità al contratto tra frontend e backend, soddisfacendo R-2-V-Ob (interfacciamento con servizi LLM tramite API).

Parser SSE: `parseSseStream`. Il parser `parseSseStream` (in `lib/sse.ts`) trasforma il `ReadableStreamDefaultReader` in un `AsyncIterable<SseEvent>`, dove `SseEvent` è definito come `{ type: 'chunk'; data: string } | { type: 'error'; message: string }`. La sua implementazione si basa sulla struttura della specifica SSE e distingue tre terminatori:

1. `data: [DONE]`: il generatore termina senza eventi di errore (successo).
2. `event: error`: il backend segnala un errore. Il parser produce un evento `{ type: 'error', message: ... }`.
3. Chiusura del reader senza nessuno dei due: il parser produce un evento di errore con il messaggio `STREAM_INTERRUPTED_MESSAGE` («La generazione si è interrotta prima di completarsi. Riprova.»). Questo messaggio è generato lato client (R-110-F-Ob).

Il parser gestisce il buffering delle righe e le terminazioni CRLF. È una **pure function**: non ha dipendenze da React o dallo store; può essere testata in isolamento.

Hook `useAiStream`. `useAiStream` incapsula il ciclo di vita di uno streaming: avvio, accumulo dei frammenti, gestione degli errori e interruzione. È descritto in dettaglio in §3.5 come esempio del pattern *Custom Hook*.

- `start<T>`: riceve una funzione che ritorna un `AsyncIterable<T>`. Crea un nuovo `AbortController`, lo salva nello store (`_setAbortController`) e avvia l'iterazione. Per ogni frammento, controlla se il controller è stato interrotto. Al termine, imposta `isGenerating` a `false` e rimuove il riferimento al controller.
- `abort`: interrompe la comunicazione tramite `controller.abort()` (R-79-F-De).
- `status`: uno stato `'idle' | 'streaming' | 'done' | 'error'` che il componente può utilizzare per mostrare indicatori visivi (R-109-F-De).

L'hook è progettato per essere usato in un solo componente alla volta (`IAiActionDialog`). L'aborto è accessibile anche da altri componenti (la `TopBar`) tramite il riferimento salvato nello store, che espone `abortStream` come azione pubblica.

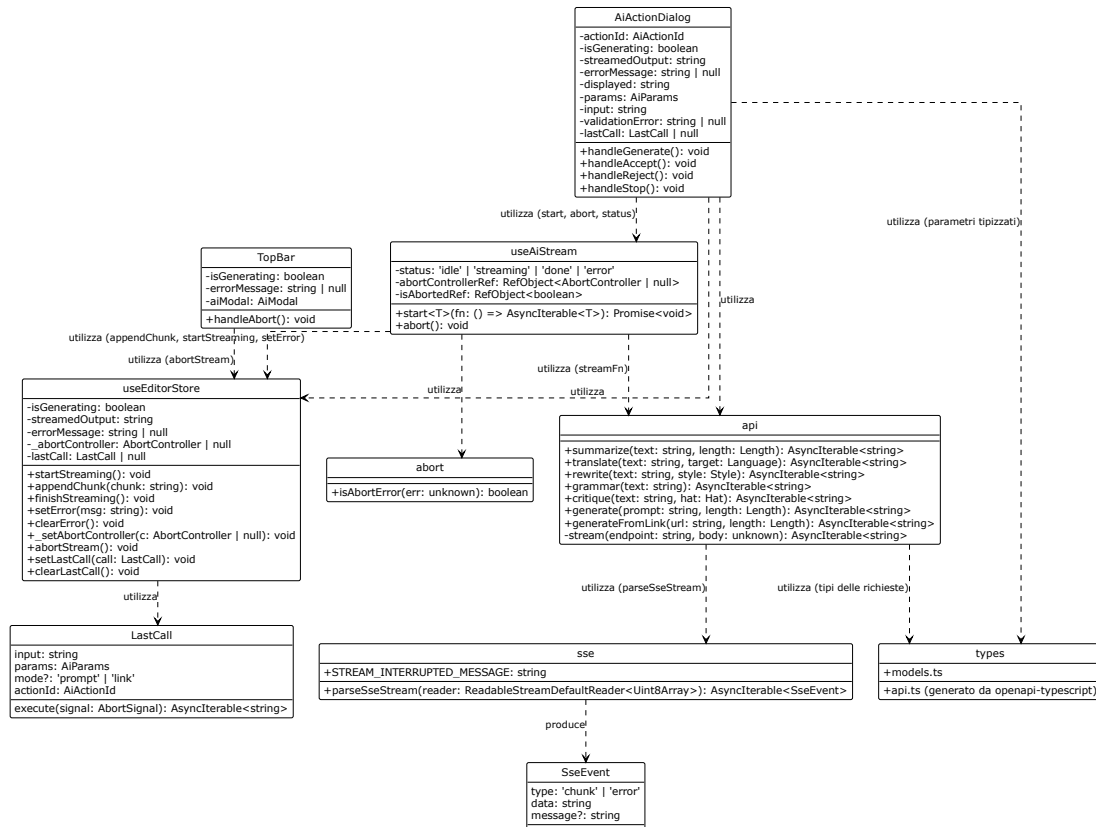


Figura 16: Diagramma UML 3.3.5 - Accesso al servizio applicativo

3.3.6 Persistenza lato client

La persistenza dei dati lato client è realizzata con due meccanismi distinti, complementari e non sovrapposti.

Persistenza automatica (Zustand persist). Come descritto in §3.3.3, `notes` utilizza il middleware `persist` di Zustand per salvare automaticamente `list` e `currentId` nel `localStorage` del browser. Il salvataggio avviene a ogni modifica dello store. Il ripristino avviene al caricamento dell'applicazione. Questo meccanismo soddisfa R-4-V-Ob (salvataggio come file di testo) e R-85-F-Ob (caricamento da file locale), garantendo la continuità della sessione.

Importazione ed esportazione (lib/fileSystem.ts). Il modulo `lib/fileSystem.ts` fornisce funzioni per l'importazione e l'esportazione delle note come file Markdown. I due rami di funzionalità:

- **Esportazione (saveNoteToFile):**
 - Ramo preferenziale: utilizza l'API `showSaveFilePicker` (Chrome/Edge).

- Ramo di fallback (Firefox): crea un link fittizio (`<a download>`) e avvia il download automatico del file `.md`. Questo ramo soddisfa R-1-V-Ob (supporto a Firefox).
- **Importazione** (`openNoteFromFile`):
 - Ramo preferenziale: utilizza l'API `showOpenFilePicker` (Chrome/Edge).
 - Ramo di fallback (Firefox): crea un `<input type="file">`, lo apre programmaticamente e legge il contenuto tramite `FileReader`. Gestisce due vie di uscita per l'annullamento: l'evento `oncancel` e il ritorno del focus alla finestra con un timer di tolleranza (`GRAZIA_ANNULLAMENTO_MS = 300ms`). Questo doppio meccanismo garantisce che la Promise non resti pendente per sempre in Firefox.
 - Restituisce una `Note` con `id` generato da `crypto.randomUUID()`, `title` derivato dal nome del file, `content` come testo letto, e `createdAt/updatedAt` al momento dell'importazione. Soddisfa R-85-F-Ob.
- **Ridenominazione** (`renameNote`): aggiorna il titolo di una nota e l'`updatedAt`. La funzione è asincrona e restituisce la nota modificata. Soddisfa R-94-F-De.

I due meccanismi non sono sovrapposti. Il middleware `persist` gestisce il salvataggio automatico e il ripristino della sessione. `fileSystem` gestisce l'operazione esplicita di importazione ed esportazione richiesta dall'utente. Questa separazione soddisfa R-1-V-Ob (supporto a Firefox) e R-4-V-Ob (salvataggio come file di testo).

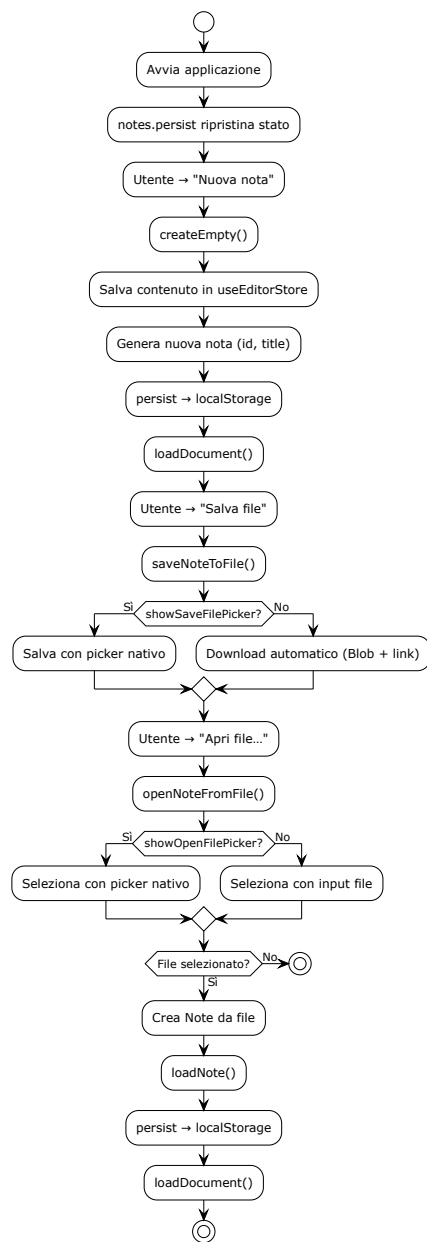


Figura 17: Diagramma UML 3.3.6 - Persistenza lato client

3.4 Architettura del backend

3.4.1 Struttura a esagono e contratti di dipendenza

3.4.2 Adattatori primari

3.4.3 Il nucleo

3.4.4 Adattatori secondari

3.4.5 Composition root e configurazione

3.5 Design pattern adottati

3.5.1 Object Adapter

3.5.2 Facade

3.5.3 Observer

3.5.4 Dependency Injection

3.5.5 Pattern valutati e non adottati

3.6 Idiomi e convenzioni di codifica

3.6.1 Generatori asincroni e propagazione dell'annullamento

3.6.2 Tipizzazione dei confini fra i livelli

3.6.3 Denominazione, struttura dei file e gestione degli errori

4 Flusso dei dati

4.1 Generazione assistita da LLM_G con risposta in streaming

4.2 Annullamento di un'operazione in corso

4.3 Generazione a partire da un collegamento

4.4 Salvataggio e caricamento di una nota

La presente sezione descrive le modalità con cui l'applicazione web persiste le note lato client, in attuazione della strategia esposta in §2.4. Come dichiarato in quella sede, i meccanismi sono due, distinti per scopo e complementari: l'accesso al file system per l'esportazione e l'importazione deliberate, e l'archivio locale del browser per la continuità della sessione. La sezione descrive entrambi, il momento e il modo in cui ciascuno viene attivato, e il recupero dei metadati dall'oggetto File.

4.4.1 Persistenza su file system

Le operazioni di salvataggio e caricamento su file sono realizzate nel modulo `lib/fileSystem.ts`, che espone le funzioni `saveNoteToFile` e `openNoteFromFile`. Entrambe adottano una strategia a due vie, selezionata a tempo di esecuzione in base alle capacità del browser ospite.

Selezione della via. Il criterio di selezione è la disponibilità effettiva dell'interfaccia nel browser in uso, non l'identificazione dell'agente utente. L'applicazione verifica direttamente la presenza dei metodi richiesti sull'oggetto globale della finestra:

- Se `showOpenFilePicker` e `showSaveFilePicker` sono presenti, viene utilizzata la **File System Access API** (disponibile su Chrome/Edge).
- In caso contrario, l'applicazione ricade sui **meccanismi universali** (input file e download link, utilizzati su Firefox e Safari).

La doppia via non è una precauzione generica: il requisito di vincolo R-1-V-Ob include Firefox fra gli ambienti di esecuzione di riferimento, quindi una realizzazione fondata sulla sola File System Access API lascerebbe scoperto uno dei tre browser prescritti.

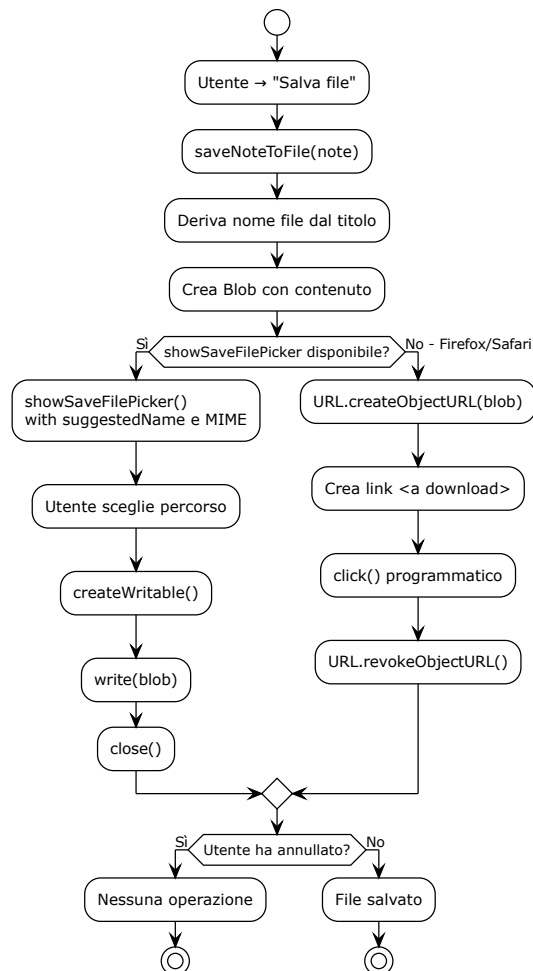


Figura 18: Diagramma UML 4.4.1-1 - fileSystem

Salvataggio su file (saveNoteToFile). La funzione riceve una `Note` e ne scrive il contenuto su file:

1. Il nome del file è derivato dal titolo della nota, con estensione `.md` (soddisfa R-4-V-Ob).
2. Il contenuto è il solo testo della nota, senza intestazioni o marcatori aggiuntivi: il file è un documento Markdown puro.
3. Se la File System Access API è disponibile, la funzione invoca `showSaveFilePicker` con il nome suggerito e il tipo MIME `text/markdown`; l'utente sceglie il percorso di destinazione; il file viene scritto tramite `FileSystemWritableFileStream`.
4. Se l'API non è disponibile, la funzione costruisce un `Blob` con il contenuto della nota, ne ricava un indirizzo temporaneo tramite `URL.createObjectURL`, lo assegna a un collegamento di scaricamento (`<a download>`) e lo attiva per via programmatica; l'indirizzo temporaneo viene rilasciato al termine dell'operazione con `URL.revokeObjectURL`.
5. L'annullamento da parte dell'utente (chiusura del selettore senza selezione) è riconosciuto tramite il controllo `eAnnullamento` su `AbortError` e trattato come esito legittimo, non come errore.

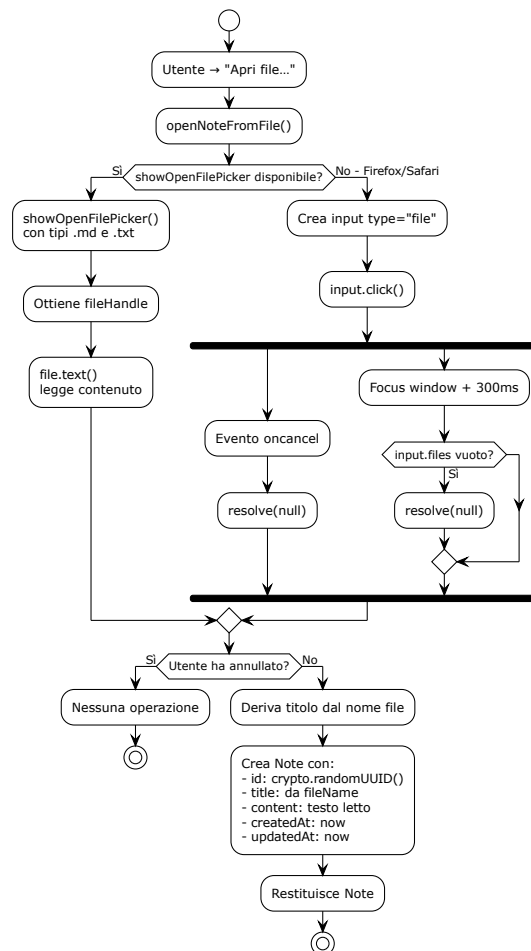


Figura 19: Diagramma UML 4.4.1-2 - saveNoteToFile

Caricamento da file (openNoteFromFile). La funzione restituisce una *Note* a partire da un file selezionato:

1. Se la File System Access API è disponibile, la funzione invoca `showOpenFilePicker` con i tipi accettati `.md` e `.txt`; ottiene un riferimento al file scelto e ne legge il contenuto testuale tramite `File.text()`.
2. Se l'API non è disponibile, la funzione crea un elemento `input` di tipo `file` con gli stessi tipi accettati; ne legge il contenuto tramite `FileReader.readAsText`. Questo ramo gestisce due vie di uscita per l'annullamento: l'evento `oncancel` (disponibile in alcuni browser) e il ritorno del focus alla finestra con un timer di tolleranza (`GRAZIA_ANNULLAMENTO_MS = 300ms`) che controlla se `input.files` è vuoto. Il doppio meccanismo garantisce che la Promise non resti pendente per sempre su Firefox.
3. La funzione restituisce una *Note* con:
 - `id`: generato da `crypto.randomUUID()`;
 - `title`: derivato dal nome del file, privato dell'estensione (`.md` o `.txt`); se il nome del file non è utilizzabile, viene usato il titolo di ripiego «Nota importata»;
 - `content`: il testo letto dal file;
 - `createdAt`: l'istante di importazione;
 - `updatedAt`: l'istante di importazione.
4. L'annullamento è riconosciuto e trattato come esito legittimo in entrambi i rami.

La derivazione del titolo dal nome del file avviene in entrambi i rami, garantendo la parità fra i due percorsi. È necessaria perché il ramo di ripiego è il percorso primario su Firefox, incluso fra i browser prescritti da R-1-V-Ob.

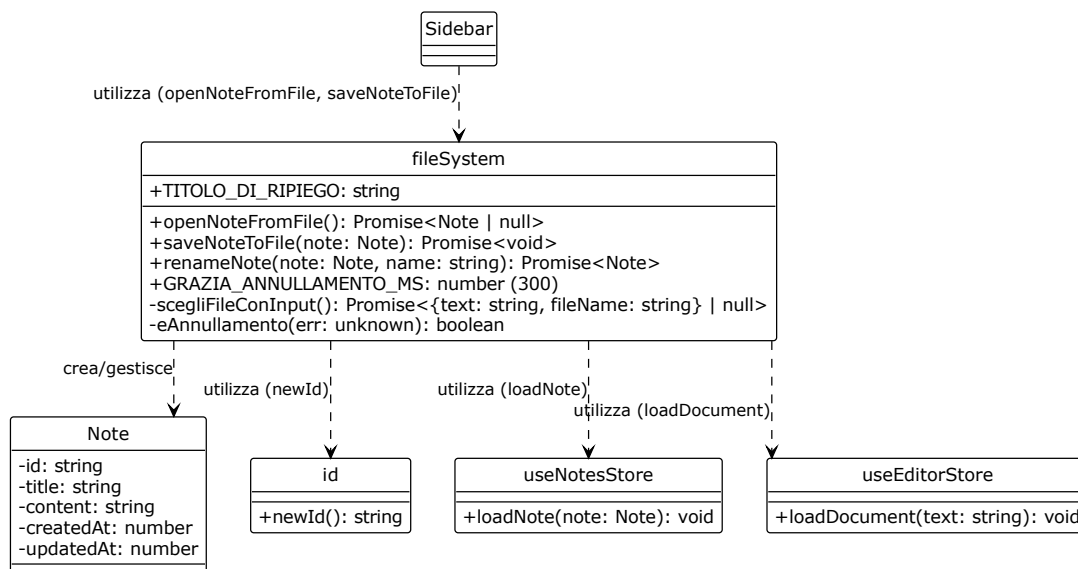


Figura 20: Diagramma UML 4.4.1-3 - openNoteFromFile

4.4.2 Continuità della sessione

La continuità della sessione è realizzata tramite il middleware `persist` di Zustand, applicato a `notes` come descritto in §3.3.3. Il meccanismo non richiede alcuna azione dell'utente.

Scrittura nell'archivio locale. Il middleware `persist` serializza lo stato di `notes` e lo riversa nel `localStorage` del browser sotto la chiave `'notes_persistence'`. Lo stato conservato comprende:

- `list`: l'elenco delle note, con `id`, `title`, `content`, `createdAt`, `updatedAt`;
- `currentId`: l'identificativo della nota correntemente selezionata.

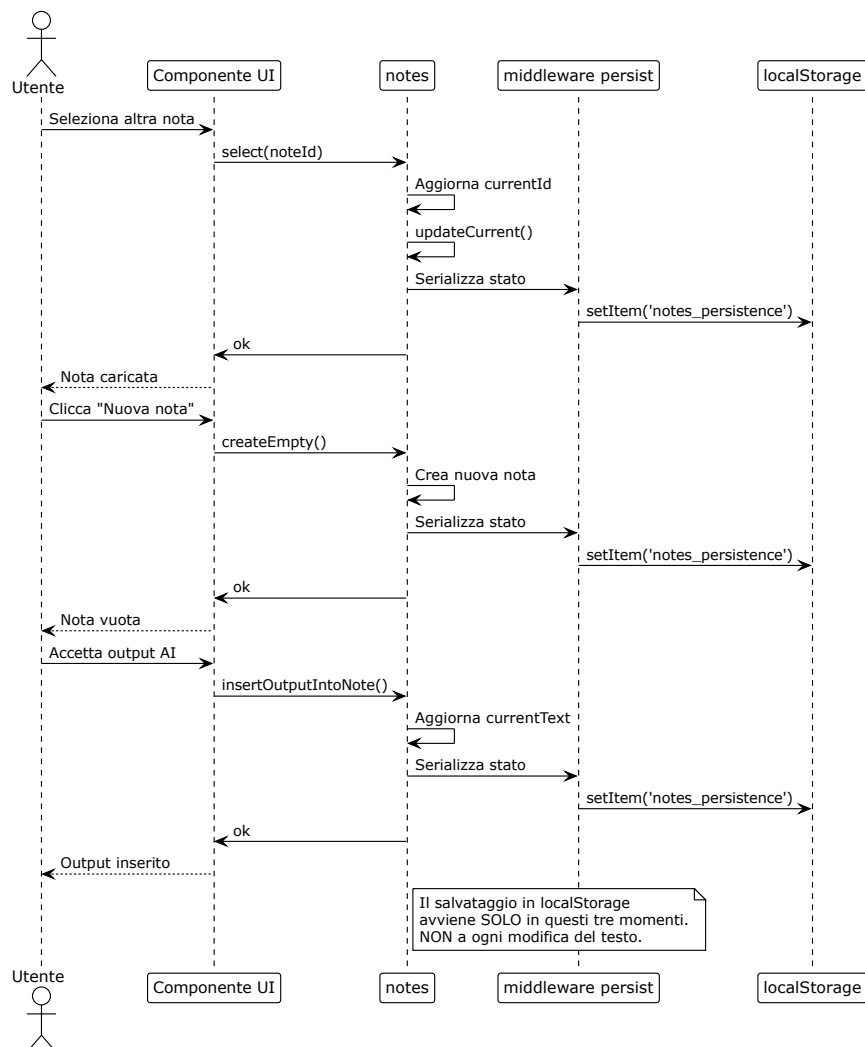


Figura 21: Diagramma UML 4.4.2-1 - salvataggio automatico in `localStorage`

Quando viene scritto il contenuto dell'editor. Va precisato quando il testo in lavorazione raggiunge l'archivio. Il contenuto dell'editor è riversato nella nota corrente nei seguenti momenti:

1. **Al cambio di nota** (`notes.select`): il contenuto corrente viene salvato nella nota che si sta abbandonando.
2. **Alla creazione di una nuova nota** (`notes.createEmpty`): il contenuto corrente viene salvato nella nota precedente prima di crearne una nuova.
3. **All'accettazione di un output AI** (UC68, `insertOutputIntoNote`): il contenuto modificato viene immediatamente scritto nella nota corrente.

Il contenuto **non** è riversato a ogni modifica del testo né alla chiusura della scheda. Le modifiche successive all'ultimo cambio di nota non sono conservate: è un limite noto della realizzazione attuale, non una proprietà del supporto. La scelta è deliberata: il costo di scrivere in `localStorage` a ogni battitura sarebbe inaccettabile per le prestazioni; il salvataggio nei momenti di transizione garantisce la sopravvivenza delle note complete, a costo di perdere le modifiche non ancora "finalizzate" da un cambio di contesto.

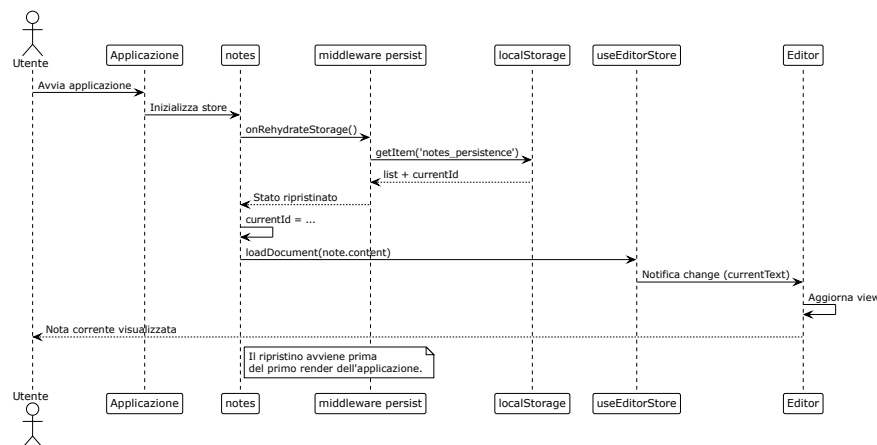


Figura 22: Diagramma UML 4.4.2-2 - lettura all'avvio

Lettura all'avvio. All'avvio dell'applicazione, il middleware `persist` rilegge lo stato dal `localStorage` prima del primo render. Durante il ripristino (`onRehydrateStorage`), viene chiamato `loadDocument` per caricare il contenuto della nota corrente nell'editor. L'applicazione riparte dallo stato in cui l'utente l'aveva lasciata: l'elenco delle note, la nota corrente e il suo contenuto sono quelli della sessione precedente.

Limiti del supporto. Il `localStorage` ha limiti propri, che vanno dichiarati:

- **Spazio:** è soggetto a una quota (tipicamente 5-10 MB per origine). Il superamento fa fallire la scrittura; l'applicazione lo tratta come condizione prevista e ripristina lo stato precedente.
- **Isolamento:** i dati sono confinati all'origine e al profilo del browser in uso. Non sono raggiungibili da un altro browser, da un altro dispositivo o da una finestra di navigazione anonima.

- **Cancellazione:** la cancellazione dei dati di navigazione rimuove i dati.

Questo meccanismo non è un sostituto della persistenza server-side: resta interamente lato client, come dichiarato in §2.4.

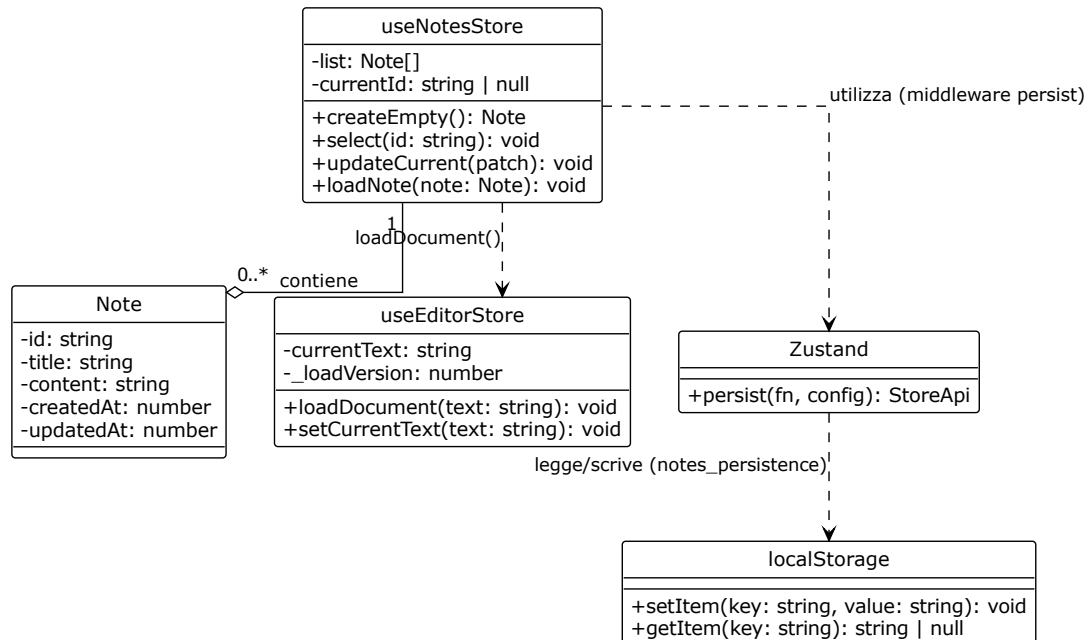


Figura 23: Diagramma UML 4.4.2-3 - continuità della sessione

4.4.3 Recupero dei metadati dall'oggetto File

Il requisito R-97-F-De (UC84) richiede la visualizzazione delle informazioni e dei metadati di una nota selezionata. Come dichiarato in §2.4, la copertura del requisito distingue due casi.

Nota creata nel prodotto. Per le note create all'interno dell'applicazione, i metadati sono posseduti e conservati:

- **id:** generato da `newId()` (da `lib/id.ts`) al momento della creazione;
- **title:** assegnato dall'utente o impostato a «Senza titolo»;
- **createdAt:** istante di creazione;
- **updatedAt:** istante dell'ultima modifica;
- **content:** il testo della nota.

Questi metadati sono affidati alla persistenza locale descritta in §4.4.2: il middleware `persist` li scrive immediatamente in `localStorage` e li rilegge all'avvio. Per queste note, R-97-F-De è già coperto.

Nota importata da disco. Per le note importate da file locale (UC76.1), il file contiene il solo testo. I metadati sono attualmente popolati come segue:

- **id:** generato da `newId()`;
- **title:** derivato dal nome del file, privato dell'estensione; se il nome non è utilizzabile, viene usato il titolo di ripiego «Nota importata»;
- **createdAt** e **updatedAt:** entrambi impostati all'istante di importazione.

Come dichiarato in §2.4, questa realizzazione sarà estesa per sfruttare gli attributi `lastModified` e `size` dell'oggetto `File`, in modo da popolare rispettivamente l'istante di ultima modifica e la dimensione del contenuto. L'istante di creazione (**createdAt**) non è ricostruibile in modo affidabile dalle informazioni disponibili; per le note importate da disco sarà presentato come «non disponibile».

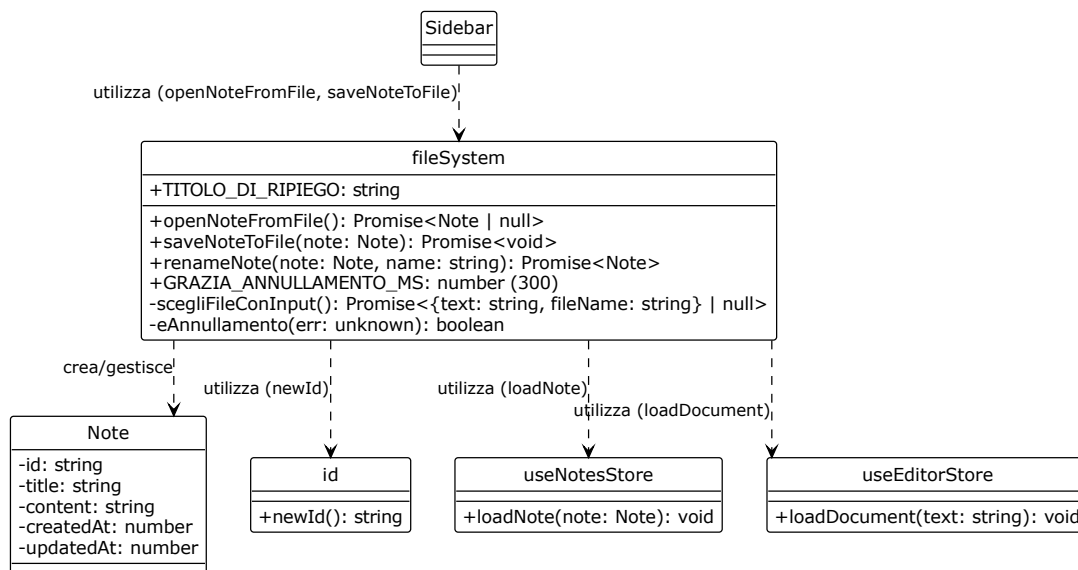


Figura 24: Diagramma UML 4.4.3 - nota creata nel prodotto vs nota importata da disco

4.5 Propagazione e presentazione degli errori

5 Tracciamento dei requisiti_G

5.1 Criteri di tracciamento

5.2 Requisiti_G funzionali

5.3 Requisiti_G di qualità

5.4 Requisiti_G di vincolo_G

5.5 Grafici riassuntivi